Weike Liu

RL,MicroDuck,MuJoCo

目录

1 安装 mujoco

2 第一个 Hello World

3 看一些真家伙

4 MuJoCo 是什么？

5 做一个机械臂

5.1 第一步：定义机器人模型（MJCF/XML 文件）

5.2 第二步：参数配置文件（YAML）

5.3 第三步：主 Python 程序（完整开发流程）

5.3.1 导入依赖与初始化

5.3.2 加载 YAML 配置（`load_config` 函数）

5.3.3 加载 MuJoCo 模型

5.3.4 进入仿真主循环（带被动查看器）

5.3.5 主循环内部（`while viewer.is_running()`）

5.3.6 退出循环与结束

5.3.7 补充说明

5.4 第四步：如何运行与效果验证

6 回到小鸭子

6.1 MicroDuck 主仓库

6.1.1 整体结构

6.1.2 MicroDuck 机器人控制的核心 robotd

6.1.2.1 robotd 的执行流程

6.1.2.2 第一部分：robotd 主流程

6.1.2.2.1 第一块：参数和配置

6.1.2.2.2 第二块：robotd init 是一条特殊路径

6.1.2.2.3 第三块：抢占 robotd 的 Unix socket

6.1.2.2.4 第四块：创建 RobotState

6.1.2.2.5 第五块：创建 Intents

6.1.2.2.6 第六块：创建 poweroff 回调

6.1.2.2.7 第七块：启动 control thread

6.1.2.2.8 第八块：主线程启动 IPC server

6.1.2.2.9 IPC server 本身怎么走？

6.1.2.2.10 IPC handler 做什么？

6.1.2.2.11 主线程什么时候退出？

6.1.2.3 第二条主线：50 Hz Control Loop

6.1.2.3.1 Control Loop 初始化阶段

6.1.2.3.2 第一块：确定当前 policy

6.1.2.3.3 第二块：创建 Safety

6.1.2.3.4 第三块：读取启动姿态

6.1.2.3.5 第四块：创建 Controller

6.1.2.3.6 第五块：建立 50 Hz ticker

6.1.2.3.7 进入 while

6.1.2.3.8 Tick 第 1 块：等待周期

6.1.2.3.9 Tick 第 2 块：读取传感器

6.1.2.3.10 Tick 第 3 块：Coast

6.1.2.3.11 Tick 第 4 块：更新 IMU / odometry / fall 状态

6.1.2.3.12 Tick 第 5 块：读取 Intent Snapshot

6.1.2.3.13 Tick 第 6 块：Safety Gate

6.1.2.3.14 Tick 第 7 块：处理一次性 power request

6.1.2.3.15 Tick 第 8 块：处理 Skill / Sit / Ground Pick

6.1.2.3.16 Tick 第 9 块：处理声音

6.1.2.3.17 Tick 第 10 块：处理 Mode Switch / Policy Reload

6.1.2.3.18 Tick 第 11 块：Shutdown / Battery Shutdown

6.1.2.3.19 Tick 第 12 块：Limp-Fall

6.1.2.3.20 Tick 第 13 块：决定是否进入 driving

6.1.2.3.21 Tick 第 14 块：处理 driving 开始 / 停止

6.1.2.3.22 Tick 第 15 块：电池电压补偿

6.1.2.3.23 Tick 第 16 块：生成 Joint Targets

6.1.2.3.24 Tick 第 17 块：Controller.step()

6.1.2.3.25 Tick 第 18 块：Theremin / Chorale / Mouth

6.1.2.3.26 Tick 第 19 块：最终进入 Safety

6.1.2.3.27 Tick 第 20 块：发布 RobotState

6.1.2.3.28 Tick 第 21 块：记录 loop health

6.1.2.4 robotd 的一些主要设计要点

6.2 robotd 中关于 ONNX 的模型声明、例化、调用

6.2.1 定义：定义一个 ONNX 模型对象

6.2.2 创建：把 `.onnx` 文件加载成 `Session`

6.2.3 `robotd` 创建它

6.2.4 执行：调用 ONNX

6.2.5 `robotd` 是怎么调用它的？

6.2.6 三个动作总结

6.3 从 ONNX 文件到程序输出：完整流程

6.3.1 先理解：ONNX 文件本身是什么

6.3.2 完整过程（四步）

7 MicroDuck 的训练

第 1 章 文档说明与训练流程总览

1.1 文档目标与使用方式

1.2 训练全流程总览

1.3 关键概念与术语

1.4 仓库与相关项目

第 2 章 环境准备

2.1 硬件要求

2.2 软件要求

2.3 克隆仓库
2.4 安装依赖
2.5 验证安装
2.6 （可选）账号登录
第 3 章 选择训练任务
3.1 查看任务注册表
3.2 任务清单
3.3 任务命名规则与 Backlash 变体
3.4 机器人与观测/动作契约
第 4 章 冒烟测试
4.1 为什么必须先冒烟
4.2 执行冒烟测试
4.3 冒烟测试失败排查
第 5 章 正式训练
5.1 启动训练
5.2 训练参数说明
5.3 训练监控与结果判断
5.4 训练时长预算
5.5 从检查点恢复训练
5.6 跨任务热启动（Warm Start）
5.7 无 GPU 方案：Hugging Face Jobs
第 6 章 评估训练结果
6.1 查看器回放（uv run play）
6.2 部署演练（infer_policy.py）
6.3 回归测试
第 7 章 导出 ONNX 与发布
7.1 导出 ONNX
7.2 验证导出文件

7.3 发布到 Hugging Face Hub

7.4 真机安装（robotctl）

7.5 渲染回放视频（可选）

第 8 章 常见问题排查

第 9 章 命令速查表

附录 A 仓库关键文件与进一步阅读

8 浏览器仿真工具（可视化理解架构运行）

9 官方文档与特性说明（补充架构设计理念）

10 RL 算法

10.1 强化学习到底在解决什么问题？

10.2 为什么需要“状态、动作、奖励”？

10.3 策略是什么？

10.4 为什么需要 Value Function？

10.5 Bellman 方程：强化学习最核心的数学结构

10.6 Q Function：为什么还需要它？

10.7 V 和 Q 的关系

10.8 最优价值函数

10.9 最优 Q 函数

10.10 Bellman Optimality Equation

10.11 为什么 Q-learning 如此重要？

10.12 TD Learning 的本质

10.13 探索与利用

10.14 从表格 Q 到神经网络

10.15 强化学习突然变成了函数逼近问题

10.16 Deep Q-Network 的损失函数

10.17 Policy Gradient：为什么可以直接学策略？

10.18 Policy Gradient 最核心的公式

10.19 Actor-Critic：价值函数和策略函数结合

10.20 Actor-Critic 的直觉

10.21 把整个强化学习数学体系串起来

10.22 如果用一句话理解整个强化学习

1 安装 MUJOCO

在 Windows 11 上，MuJoCo 早已是纯 Python 包，不需要申请许可证或配置复杂环境变量。

打开命令提示符(cmd)或 PowerShell。

执行安装命令 bash：pip install mujoco

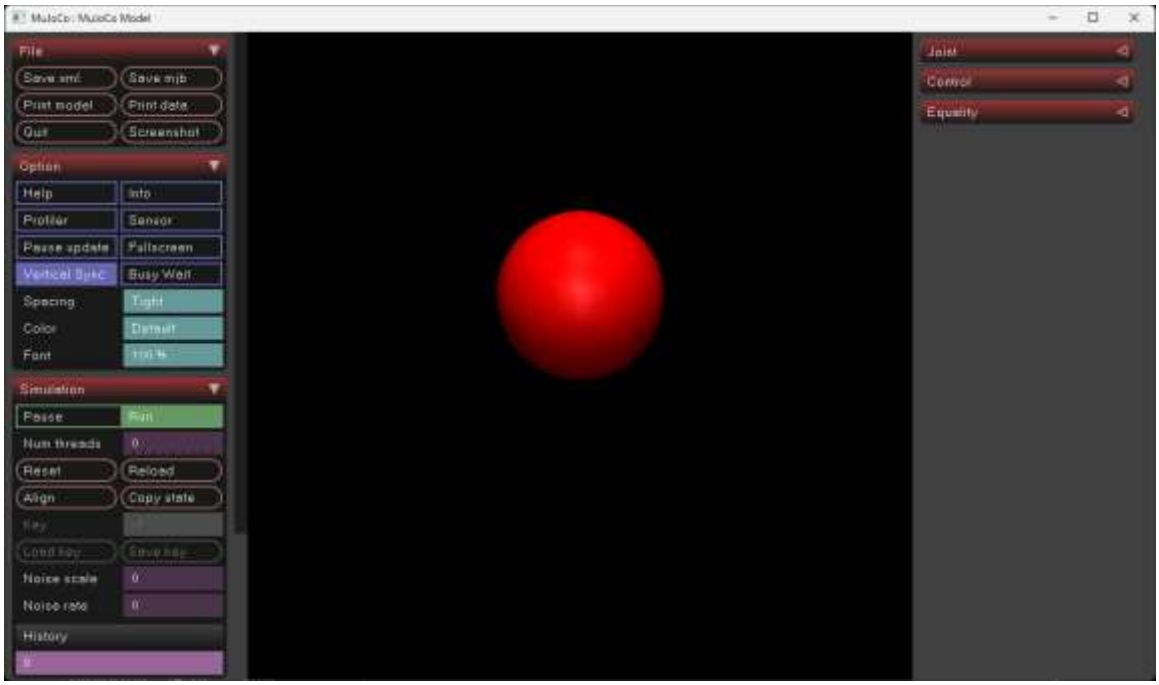
这个命令会自动装好核心库和渲染依赖。

安装后可用 python -c "import mujoco; print(mujoco.__version__)"验证。

2 第一个 HELLO WORLD

HelloWorld.py

运行：python HelloWorld.py，立即出现：



3 看一些真家伙

安装 yaml、torch 等等。

Pip install pyyaml

Pip install torch

git clone https://github.com/limxdynamics/humanoid-description.git

进入目录，然后：

python -m mujoco.viewer --mjcf=HU_D04_description/xml/HU_D04_01.xml

此外，还有宇树机器人：git clone https://github.com/yueruyin/robot-unitree-g1.git

4 MuJoCo 是什么？

MuJoCo 的核心是一个用 C 语言写成的物理引擎库，Python 只是其中最流行、最方便的一种调用方式。

具体来说，你有以下 4 种使用途径：

- Python 接口（最常用的）：通过 pip install mujoco 安装。这是官方维护的 Python 绑定，把 C 库包装成了 Python 模块。你现在运行脚本、查看模型用的都是这种方式。
- C/C++ 原生调用：如果你是做机器人底层开发或嵌入式的，可以直接下载 MuJoCo 的 C 库（.dll/.so/.dylib），在 C/C++ 代码里直接调用它的物理计算函数。
- 命令行工具：安装 Python 包后，会自动附带命令行工具。比如你之前用过的 python -m mujoco.viewer --mjcf=xxx.xml，本质上是调用了内置的可视化程序。
- 其他语言的绑定：社区还提供了 Julia（MuJoCo.jl）、Rust（mujoco-rs）甚至 MATLAB 的接口，覆盖了大部分主流编程语言。

总结：Python 是入门和做强化学习（RL）的首选，但 MuJoCo 本身非常“通用”，不绑定任何语言。

1. 核心定位：库（Library）vs.平台（Platform）

MuJoCo 是“库”：它就像 OpenCV（计算机视觉库）或 FFmpeg（音视频库）。它是一套底层计算工具，没有“主界面”。你的代码是“总司令”，调用它的 API 来计算物理、接触力、关节角度，然后你把计算结果用别的工具画出来。它完全围绕代码工作。

Isaac Sim 是“平台/应用”：它就像 Unity 或 Unreal 引擎。它是一个巨大的图形化集成环境。你绝大部分时间在 GUI 里拖拽模型、搭建场景、设置光源，点按钮运行仿真。代码（Python）只是用来辅助 GUI 操作或编写复杂逻辑的“插件”。

2. 关于 GUI 的“陷阱”（很多人误解的地方）

你可能会问：“可是我用 python -m mujoco.viewer 不是也弹出一个窗口（GUI）了吗？”

注意区分：MuJoCo 自带的这个窗口，本质上是一个“调试监视器”，而不是“开发平台”。

Isaac Sim 的 GUI：你是在它里面工作（建场景、改物理参数、看渲染效果）。

MuJoCo 的 viewer：你只是把它当显示器（用来看看机器人到底跑得怎么样，腿有没有抖，有没有摔倒）。一旦你关掉这个 viewer，你的 Python 脚本依然可以在后台计算物理数据（比如记录日志或训练 AI），而 Isaac Sim 一旦关掉 GUI，整个仿真环境就结束了。

5 做一个机械臂

5.1 第一步：定义机器人模型（MJCF/XML 文件）

MuJoCo 使用 MJCF（MuJoCo XML） 格式描述机器人。这个文件定义了机械臂的几何形状、关节、质量、颜色等。建一个 two_link_arm.xml 文件。

5.2 第二步：参数配置文件（YAML）

YAML 在这里负责存放“控制算法的参数”，避免写死在 Python 代码里：config.yaml。

5.3 第三步：主 PYTHON 程序（完整开发流程）

run_arm.py，这是核心代码。把加载模型、读取 YAML、控制循环、可视化全部整合在一起。

程序有很好的注释，非常易读。

这个 Python 程序（`run_arm.py`）实现了一个基于 MuJoCo 物理引擎的两连杆机械臂轨迹跟踪仿真。它通过 YAML 配置文件设定关节目标轨迹，利用 XML 模型中内置的 `position`执行器（自带 PD 控制）直接下发期望角度，从而驱动机械臂运动。下面按执行顺序逐步讲解其流程：

5.3.1 导入依赖与初始化

导入 `time`、`numpy`、`yaml`、`mujoco` 及其 `viewer` 子模块。这些库分别用于时间控制、数值计算、配置文件解析、物理模拟和可视化。

5.3.2 加载 YAML 配置（`load_config` 函数）

尝试打开 `config.yaml` 文件并解析为字典。

- 若文件不存在，则使用默认参数（仅包含轨迹幅值、频率和相位差），并给出警告提示。

- 配置内容用于后续生成两个关节的目标角度轨迹。

5.3.3 加载 MuJoCo 模型

通过 `mujoco.MjModel.from_xml_path("two_link_arm.xml")` 读取 XML 模型文件。

若文件加载失败，程序打印错误并退出。

接着创建 `MjData` 对象，用于存储仿真状态（位置、速度、控制量等）。

5.3.4 进入仿真主循环（带被动查看器）

使用 `mujoco.viewer.launch_passive()` 启动一个被动（非阻塞）可视化窗口，并传入模型和数据。

记录起始时间 `start_time`，用于计算仿真经过的时间 `elapsed`。

5.3.5 主循环内部（`while viewer.is_running()`）

每次迭代执行以下步骤：

a. 计算期望关节角度

- 根据配置中的 `amplitude1`、`freq1` 等参数，利用正弦函数生成两个关节的目标角度：

  - `target1 = A1 * sin(2π * f1 * t)`

  - `target2 = A2 * sin(2π * f2 * t + ϕ2)`

- 这里 `t` 为仿真运行时间（物理时间，非 CPU 时间）。

b. 直接下发目标角度

- 将计算出的 `target1` 和 `target2` 赋值给 `data.ctrl[0]` 和 `data.ctrl[1]`。

- 关键点：XML 模型中关节执行器类型为 `<position>`，其内部已经定义了 `kp`（比例增益）和 `kv`（速度增益），因此 MuJoCo 会自动将角度误差转化为力矩，**无需在 Python 中手动计算 PD 控制律**。

c. 步进物理引擎

- 调用 `mujoco.mj_step(model, data)` 推进仿真一步，更新位置、速度、力矩等所有状态。

d. 同步可视化

- 调用 `viewer.sync()` 将当前数据渲染到显示窗口。

e. 控制仿真速度

- 使用 `time.sleep(model.opt.timestep)` 让程序休眠一个物理步长的时间（通常为几毫秒），从而保证仿真运行速度与真实时间大致对齐（实时仿真），避免循环过快。

5.3.6 退出循环与结束

- 当用户关闭查看器窗口或按下停止键时，`viewer.is_running()` 变为 `False`，循环终止。

- 打印 `"仿真结束"`，程序退出。

5.3.7 补充说明

- 整个流程是 **“配置 → 模型加载 → 实时轨迹生成 → 直接位置控制 → 物理模拟 → 渲染”** 的闭环。

- 该设计将底层控制（PD 参数）置于 XML 中，上层 Python 只负责轨迹规划，实现了控制逻辑与物理仿真的分离，便于调试和参数调整。

- 仿真的实时性依赖于 `time.sleep`，但实际物理步长由 `model.opt.timestep` 决定（通常固定），而 `sleep` 仅用于节流，若计算机性能不足可能导致仿真变慢，但不会影响物理精度。

5.4 第四步：如何运行与效果验证

执行脚本：python run_arm.py

看到：一个绿色+蓝色的机械臂，底座旋转（关节 1），同时大臂上下摆动（关节 2），末端橙色小球画着优雅的“8”字轨迹。你可以用鼠标拖拽旋转视角，右键平移，滚轮缩放。

透过这个例子，理解 MuJoCo 开发的“四步法”

环节	文件/代码	作用解析
① 模型定义	*.xml (MJCF)	这是机器人的“物理蓝图”。定义了几何形状、质量、关节类型（hinge/ball/free）、摩擦、颜色等。MuJoCo 的精华全在 XML 的调参里。
② 参数配置	*.yaml	这是“控制算法的调参表”。把频率、幅度、增益拿出来，方便做实验时反复修改，避免频繁改 Python 代码。
③ 核心逻辑	Python 脚本	负责加载（模型+YAML）、计算（目标轨迹）、执行（赋值

		data.ctrl）、步进（mj_step）。这是你写算法的战场。
④ 可视化循环	viewer.launch_passive	被动循环让你掌握主动权——先算控制，再步进物理，最后刷新画面。这比 launch（全自动）更适合做算法开发。

进阶小贴士

调整参数：修改 config.yaml 里的 amplitude 或 freq，保存后重新运行，观察机械臂摆动变化，无需改 Python。

导出数据：在循环里加一行 print(data.body('tip').xpos)，就能实时打印末端执行器的三维空间位置，用于分析轨迹。

改成关节空间跟随：如果你想让他保持某个固定姿势，把 target1 = 0.5 写死就行了。

这个框架几乎涵盖了所有 MuJoCo 仿真项目的基础结构。你可以把 two_link_arm.xml 替换成 Unitree G1 的模型，把正弦波替换成强化学习策略，就升级成进阶项目了。

6 回到小鸭子

如果对 ONNX 等内容不够了解，可以先看 6.3。

MciroDuck 有这样的一些官方开源仓库：

- 官方主仓库，Microduck 的核心仓库，包含 SDK 和机器人本体软件。

地址：`https://github.com/pollen-robotics/microduck/tree/main`

开源协议：Apache-2.0，这意味着你可以自由地使用、修改、复制和分发代码。包括：

商业使用：可以将代码用在商业软件或内部项目中。

闭源分发：将代码集成到你自己的产品中，无需开源你自己的源代码。

再许可：可以将整个作品（包含 Apache-2.0 代码）以其他协议分发。

你需要遵守的义务：

保留声明：分发代码时，必须保留原始代码中的版权声明、许可声明和 NOTICE 文件中的所有 attribution 信息。

标记修改：如果你修改了文件，需要明确标注出修改的地方。

附带协议：再分发时，需要附上一份 Apache-2.0 的协议副本。

- 官方强化学习仓库，这个仓库专注于强化学习与仿真到现实（sim-to-real）的部署工具。地址：`https://github.com/pollen-robotics/microduck_rl`
- 官方相关工具

Microduck Studio：一个本地的 Web 开发工作室，用于调试、可视化和编排 Microduck 的工作流。地址：`https://github.com/microai-lab/microduck-studio`

- 此外，还有第三方复刻项目，是一个由社区发起的第三方复刻项目，旨在从官方公开的仿真模型和源码中还原出装配图、CAD 装配体等。地址：`https://github.com/fanhao375/microduck-replica`

需要注意的是，Microduck 的软件是开源的（Apache-2.0），但硬件本身并不开源。

6.1 MICRODUCK 主仓库

地址：https://github.com/pollen-robotics/microduck/tree/main。这个仓库里面是运行在 MicroDuck 里面的 RK3566 处理器上的软件，MicroDuck 有时候把它叫做“runtime”运行时软件。

Microduck 不是一个“单一程序”，而是一套运行在机器人 RK3566 Linux 板子上的 Rust 多进程机器人操作系统/运行时。

其中核心是 robotd，负责真正的机器人闭环控制；其他 daemon[1] 围绕它分别负责配置、蓝牙、手柄、摄像头/远程通信、ToF、软件更新。所有控制类服务通过 Unix Socket + JSON-RPC 2.0/NDJSON 通信。

---

1 在 Linux 中，daemon（守护进程） 就是一直在后台运行、默默提供服务、没有交互界面的特殊进程。你可以把它理解为电脑里的“幕后工作人员”或“系统服务员”：你不需要直接跟它说话（没有图形界面或命令行交互）；它随时待命（一直在内存中运行）；有请求时它就工作（响应客户端的调用）；没有请求时它就闲着（占用极少的 CPU 资源）。一个容易混淆的点：

daemon（守护进程）：后台运行，无交互，提供系统服务。

而 service（服务）：更偏“功能层面”的叫法。守护进程是服务的具体实现进程，两者常混用。比如“SSH 服务”由 sshd 这个守护进程提供。

如何创建自己的守护进程（简要），如果你用 C 语言编写，标准步骤是：

fork() 创建子进程，父进程退出；子进程调用 setsid() 创建新会话；关闭文件描述符（标准输入/输出/错误）；切换工作目录到 /；设置文件权限掩码 umask(0)等等。

不过现在更推荐直接编写 systemd service 单元文件，让 systemd 帮你管理，省去这些底层细节。

6.1.1 整体结构

仓库本身是一个 Rust workspace，目前 Cargo workspace[2]有 19 个 member，包括 daemon、控制算法库、CLI、IPC 协议、工具等。

它有这些“入口”：

- Linux/systemd 服务入口

机器人启动后真正常驻运行的程序主要是：

入口	类型	核心作用
robotd	daemon	机器人运动控制核心
configd	daemon	WiFi、机器人身份、配对、系统配置
updaterd	daemon	软件/模型更新、验证、回滚
btd	daemon	BLE API 入口
padd	daemon	游戏手柄入口
mediad	daemon	摄像头/麦克风/WebRTC/远程 API
tofd	daemon	ToF 深度传感器

这 7 个是整个运行时最核心的服务划分。架构文档也明确说明 systemd 是它们的 supervisor，负责生命周期、崩溃重启、启动顺序和 watchdog。

---

2 Cargo Workspace 是 Rust 生态中 Cargo 工具的专属专有名词，是 Rust 官方为管理多关联包项目专门设计的核心特性，在其他编程语言的构建体系中没有完全对等的原生同名概念。核心定义：

Cargo Workspace 是一组被协同开发、相互关联的 Rust 包（crate）的集合，所有成员包会共享同一份 Cargo.lock 依赖锁文件和同一个 target 编译输出目录，从根本上避免了多包开发时的依赖版本碎片化和重复编译问题。

核心特性：

统一管理多包：可以将一个大型项目拆分为多个职责单一的独立 crate，所有 crate 都在同一个工作空间下协同维护，典型案例就是 Rust 官方的 rustc 编译器仓库，整个项目就是一个巨型 Cargo Workspace，管理着近 50 个相互依赖的子 crate。

共享配置与依赖：支持在根目录的 Cargo.toml 中通过 [workspace.dependencies] 定义全局共享依赖，所有子 crate 可以直接继承这些依赖版本，无需重复编写配置，大幅降低依赖版本冲突的概率。

灵活的构建控制：既可以在根目录执行 cargo build 一次性编译所有成员包，也可以通过 -p 包名 参数单独构建、测试或发布指定的某个子 crate，兼顾整体构建效率和单包迭代的灵活性。

两种清单模式：分为虚拟清单模式（根目录的 Cargo.toml 仅包含 [workspace] 配置，本身不作为独立 crate）和根目录作为成员包的模式，适配不同的项目组织需求。

- 用户/开发者 CLI 入口

它另外有两个非常重要的 CLI（Command Line Interface）：robotctl、duckctl。

✧ Robotctl：这是直接在机器人 Linux 板子上执行的管理 CLI。

它可以有这样的一些操作操作：

robotctl version

robotctl health

robotctl monitor

robotctl configure

robotctl policy ...

robotctl robot ...

robotctl pad ...

robotctl update ...

robotctl wifi ...

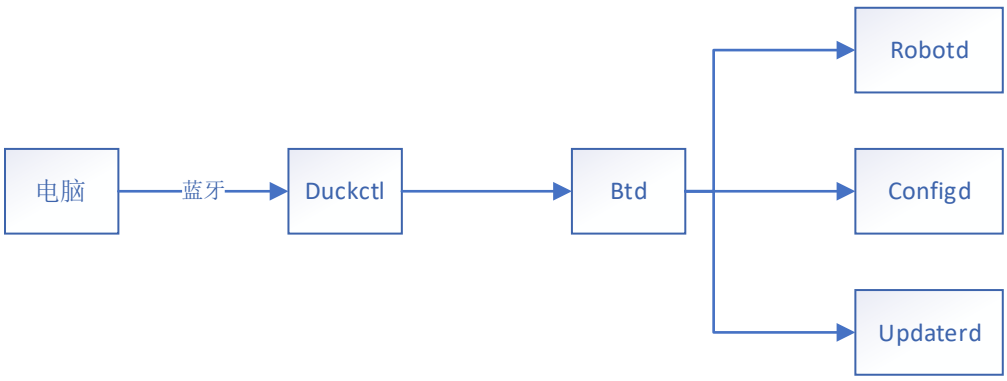
robotctl system ...

robotctl voice ...

官方 cheat sheet 把 robotctl 定义成机器人上的主要控制/诊断入口。

✧ Duckctl，这是机器人外面的电脑上的 CLI。它通过 Bluetooth 找到 Microduck，然后发送和机器人内部相同的 JSON-RPC 请求。

所以：



代码注释明确写着，duckctl 是“phone app 的 stand-in”，只用于开发，不部署到机器人上。

6.1.2 MicroDuck 机器人控制的核心 robotd

它是真正让鸭子动起来的实时控制 daemon。robotd 执行 50Hz 控制 loop，驱动 15 个舵机，并运行神经网络 policy。robotd 是唯一能够控制电机的 daemon，其原因后面再讲。MicroDuck 用 rust 语言编写，因此许多概念涉及 rust 语言的概念，如下面的 slot、tokio 等等。

6.1.2.1 robotd 的执行流程

MicroDuck 的 robotd 是用 rust 语言写的，它的入口是：

```
#[tokio::main]

async fn main() -> ExitCode
```

程序启动后，main()

1. 读取 robotd.toml
2. 创建/run/robotd.sock
3. 创建 RobotState + Intents
4. 启动 control thread
5. 当前主 tokio runtime 跑 IPC server

非常值得注意的一点：第 4 步启动的 Control loop 是一个线程，不是和第 5 步 IPC 共用一个 runtime。代码专门创建了一个：

```
OS thread: control
    |
    └── current-thread tokio runtime
            |
            └── 50 Hz loop
```

原因是 Dynamixel serial I/O 是 blocking（阻塞）的。如果把 control loop 和 IPC 放在普通 tokio runtime：

```
IPC request
    ↓
tokio worker
    ↓
blocking serial read（阻塞式的串口读）
    ↓
```

整个 worker 被占住，IPC 循环中的其他任务就可能影响控制周期。

因此把 main.rs 看成一个“主流程 + 控制循环”，重点只抓两个主线：

1. robotd 主流程：进程启动后，怎样建立服务、创建共享状态、启动控制线程、提供 IPC。
2. 50 Hz control loop：控制线程里面每一个 tick 按什么顺序执行。

下面以当前 main 分支代码为准。robotd/src/main.rs 当前约 7600 行，但真正决定运行时主流程的核心区域集中在 main()、spawn_control_thread() 和 control_loop()。

6.1.2.2 第一部分：robotd 主流程

main() 是真正的进程入口。代码从：

#[tokio::main]

async fn main() -> ExitCode 开始。

这里实际上做的是一系列初始化和组装，而不是控制机器人。按代码顺序：

main()

├── 1. 解析命令行参数

├── 2. 初始化 logging

├── 3. 加载 robotd.toml

├── 4. 处理 --port / --no-policy

├── 5. 如果是 `robotd init`，走独立 init 流程

├── 6. claim Unix socket

├── 7. 创建 RobotState

├── 8. 创建 Intents

├── 9. 创建 poweroff callback

├── 10. 创建 control thread

├── 11. 启动 IPC server

└── 12. 等待 server / SIGTERM

所以 main() 本质上是把 robotd 的运行环境组装起来，然后把两个主要执行实体启动起来：control thread + IPC server。

6.1.2.2.1 第一块：参数和配置

Params 是静态配置。比如：bus port、control period、policy、safety、deadman、gain、limp fall、audio、mode 等等。而后面运行过程中不断变化的东西，不放在 Params

里，而放到：RobotState、Intents、Controller 这三个不同层次里。

6.1.2.2.2 第二块：robotd init 是一条特殊路径

robotd init 是一条 CLI 命令，它的执行是一条单独的路径，完成初始化后，机器人是“建立机器人已知姿态”，而不是正常的持续控制服务。

6.1.2.2.3 第三块：抢占 robotd 的 Unix socket

然后才允许启动后面的 control thread。代码明确要求在启动可能接触电机的线程之前先拥有 endpoint。

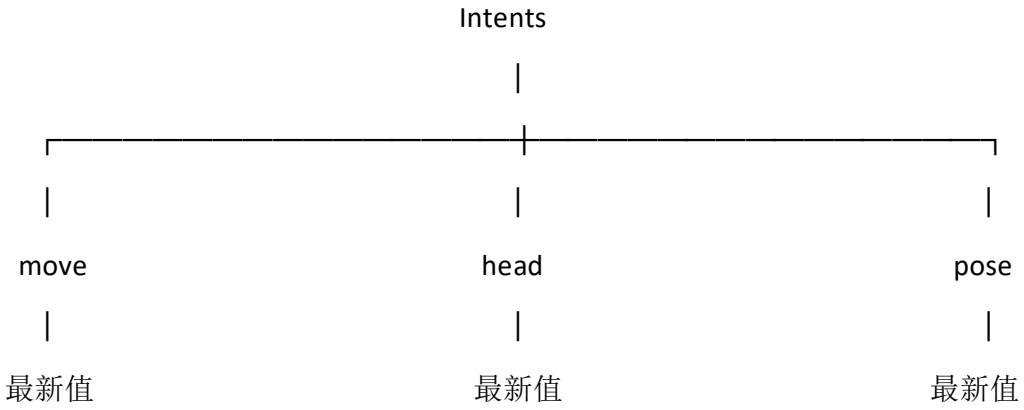
6.1.2.2.4 第四块：创建 RobotState

接下来：let state = Arc::new(RobotState::new(…));这里的 RobotState 很重要。它不是“机器人当前所有状态的 Mutex”。它主要是：control loop → IPC 的状态发布通道。

6.1.2.2.5 第五块：创建 Intents

接下来：let intents = Arc::new(Intents::new());

这个对象是 IPC 和 control loop 之间的命令交换区。注意，它不是普通的 command queue。更像：



比如客户端不断发送：

robot.move(vx=0.1)

robot.move(vx=0.2)

robot.move(vx=0.3)

control loop 并不是逐条消费：

0.1

0.2

0.3

而是在 tick 时：

let snapshot = intents.snapshot();

拿到当前这一刻的命令快照。

所以：

IPC 产生 command

↓

Intents

↓

20ms 时刻 snapshot

↓

当前控制周期使用

这是一个非常关键的架构点。

6.1.2.2.6 第六块：创建 poweroff 回调

然后：let poweroff: PowerOff = Arc::new(...)。它本身不参与普通控制。它只是给 control loop 一个：“如果决定关机，真正执行 systemctl poweroff”的能力。

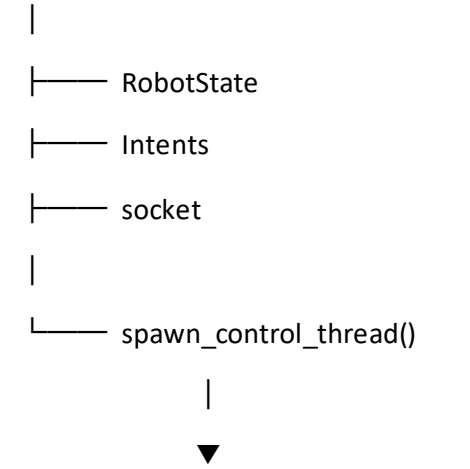
所以 control loop 不直接依赖 shell/systemd 逻辑，而是收到一个：PowerOff callback。

这也方便测试。

6.1.2.2.7 第七块：启动 control thread

然后：let control = spawn_control_thread(…)这是整个 main() 的分界点。到这里：

main thread

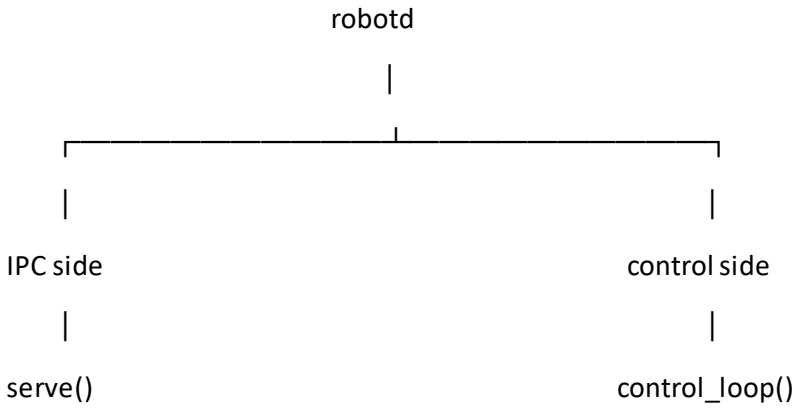


control thread

注意：真正的机器人控制从这里开始进入另一个执行上下文。

6.1.2.2.8 第八块：主线程启动 IPC server

control thread 启动后，主线程继续：let serving = serve(...)。也就是说：



然后：

```
tokio::select! {
    result = serving => ...
    _ = shutdown() => ...
}
```

主线程就在这里等待。

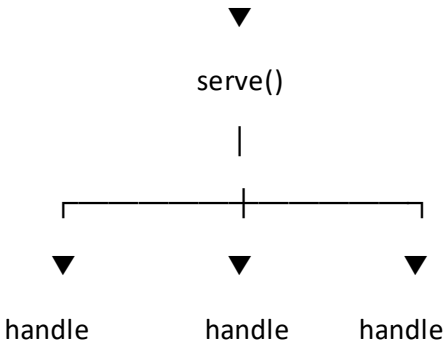
6.1.2.2.9 IPC server 本身怎么走？

serve() 很简单：

```
loop
 ├──── listener.accept()
 ├──── 得到 UnixStream
 └──── tokio::spawn(handle(...))
```

也就是说：

```
client A ────┐
client B ────┼──── Unix socket
client C ────┘
```



每个 client 都可以独立处理。代码就是 accept → clone state/intents → tokio::spawn(handle)，即每个 client 都由各自的 handle 来处理。

6.1.2.2.10 IPC handler 做什么？

handle() 收到 JSON-RPC/NDJSON 请求。它有两类事情：

第一类：查询

例如：

robot.health

robot.state

robot.policies

robot.safeToRestart

这些主要读：RobotState

第二类：控制

例如：

robot.move

robot.head

robot.init

robot.relax

robot.do

robot.setMode

...

这些不是直接控制电机。而是由 request 产生 intent 去 control thread。因此最终都要等 control loop 的下一 tick。

6.1.2.2.11 主线程什么时候退出？

当：SIGTERM 或者：IPC server 出错，主流程结束。然后：

state.shutdown.store(true, ...)

通知 control loop：“停止。”，然后：

control.join()

等待 control thread 真正退出。

最后：

remove_file(socket)删除 Unix socket。

这里还有一个非常重要的细节：不是直接 kill control loop。代码要求：

shutdown = true

↓

control loop 自己结束当前 tick

↓

退出

↓

join

原因是不能在 Dynamixel bus transaction 中间强行 abort。

6.1.2.3 第二条主线：50 Hz Control Loop

这才是 robotd 的核心。入口：async fn control_loop<T: RobotIo>(...)，代码从约 1600 行开始。它启动以后不是马上：

while true

policy

而是先建立控制环境。

6.1.2.3.1 Control Loop 初始化阶段

按照代码顺序：

control_loop()

├── 1. 准备 policy 配置

├── 2. 清理不可加载的 override

├── 3. resolve policy

├── 4. 创建 Safety

├── 5. 读取并采用 startup pose

├── 6. 创建 Controller

├── 7. 创建 50Hz ticker

└── 8. 初始化控制循环状态

6.1.2.3.2 第一块：确定当前 policy

6.1.2.3.3 第二块：创建 Safety

6.1.2.3.4 第三块：读取启动姿态

6.1.2.3.5 第四块：创建 Controller

6.1.2.3.6 第五块：建立 50 Hz ticker

然后：let mut ticker = tokio::time::interval(period);正常情况下：period = 20ms，然后：

```
ticker.set_missed_tick_behavior(
    MissedTickBehavior::Skip
);
```

这点非常重要。它意味着：

```
tick 1
tick 2
tick 3    ← 这次太慢，错过
tick 4
```

不会补：

```
tick 3
tick 3
```

而是直接进入最新时间点。控制系统不希望堆积旧 command。代码也明确说明选择 Skip 是为了避免 backlog 和时间漂移。

6.1.2.3.7 进入 while

代码：

```
while !state.shutdown.load(...) {
    ticker.tick().await;
```

这就是机器人真正的：50Hz 主循环。下面一定要严格按照代码顺序理解。

6.1.2.3.8 Tick 第 1 块：等待周期

每次：ticker.tick().await; 等到下一个控制周期。然后：let tick_start = Instant::now();记录本 tick 起点。

6.1.2.3.9 Tick 第 2 块：读取传感器

6.1.2.3.10 Tick 第 3 块：Coast

如果读传感器失败。代码：let sensors = coast.sample(fresh);含义：

fresh read 成功

↓

使用 fresh

fresh read 失败

↓

短时间使用 last good sample

最多 COAST_TICKS = 3 也就是大约：60 ms

6.1.2.3.11 Tick 第 4 块：更新 IMU / odometry / fall 状态

6.1.2.3.12 Tick 第 5 块：读取 Intent Snapshot

6.1.2.3.13 Tick 第 6 块：Safety Gate

这里解决：“用户想让机器人怎么走”是否已经超时、是否应该被限制。

6.1.2.3.14 Tick 第 7 块：处理一次性 power request

6.1.2.3.15 Tick 第 8 块：处理 Skill / Sit / Ground Pick

ground_pick、sit_toggle、skill requests 这些请求不是直接产生 joint target。它们只是改变：Controller 的内部状态。例如：

robot.do kick

↓

controller.start_skill()

↓

active = kick

然后真正的 target 要到后面的：controller.step()才产生。这一点非常重要：IPC request 是“改变控制器状态”，不是“直接输出电机目标”。

6.1.2.3.16 Tick 第 9 块：处理声音

6.1.2.3.17 Tick 第 10 块：处理 Mode Switch / Policy Reload

6.1.2.3.18 Tick 第 11 块：Shutdown / Battery Shutdown

6.1.2.3.19 Tick 第 12 块：Limp-Fall

6.1.2.3.20 Tick 第 13 块：决定是否进入 driving

这是整个 loop 最重要的一个条件判断：

```
let driving =
    snapshot.enabled
    && bringup == Bringup::Ready
    && controller.is_some()
    && !in_limp_fall
    && sensors.is_some()
    && imu_warm
    && !powered_off;
```

只有满足所有条件，policy 才真正拥有机器人。

6.1.2.3.21 Tick 第 14 块：处理 driving 开始 / 停止

6.1.2.3.22 Tick 第 15 块：电池电压补偿

目的是尽量保持 servo 的有效控制响应。

6.1.2.3.23 Tick 第 16 块：生成 Joint Targets

6.1.2.3.24 Tick 第 17 块：Controller.step()

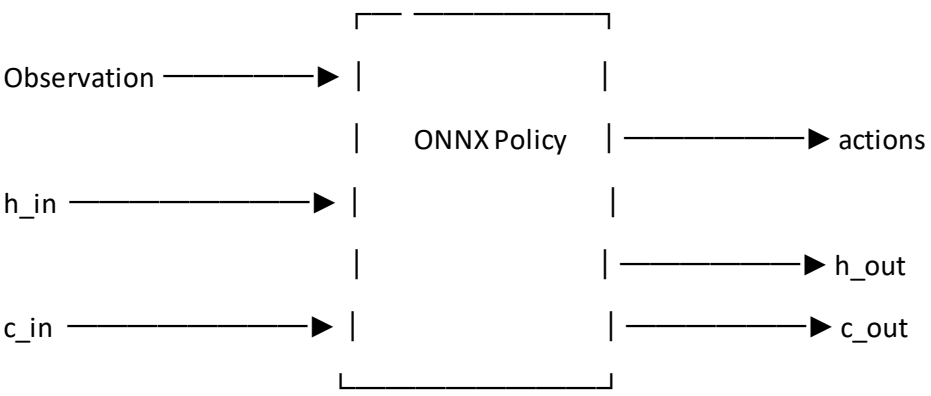
在 robotd 的 control loop 里面，当机器人处于真正 driving 状态时，会进入：

```
controller.step(
    sensors,
    &command,
    body_active,
    period.as_secs_f64(),
    scale_mult,
```

)

也就是说：control loop 本身不直接调用 ONNX。它把传感器、command 等交给 Controller。而 Controller 才是控制算法这一层。robotd/src/control.rs 文件开头甚至直接写明：Controller 位于 RobotIo::read 和 safety apply 之间，只做计算，自己没有 IO handle，因此不能直接控制电机。

所以模型实际上是：



Policy 并不是直接学习“绝对关节角度”，而是学习相对于默认/home pose 的动作偏移。

6.1.2.3.24.1 Controller 第 1 块：选择哪个 Network

6.1.2.3.24.2 Controller 第 2 块：构造 Observation

6.1.2.3.24.3 Controller 第 3 块：Action → Joint Target

6.1.2.3.24.4 Controller 第 4 块：Low-pass

6.1.2.3.24.5 Controller 第 5 块：更新自己的状态

6.1.2.3.25 Tick 第 18 块：Theremin / Chorale / Mouth

6.1.2.3.26 Tick 第 19 块：最终进入 Safety

Policy

↓

targets

↓

表达层修改

↓

Safety.apply()

↓

RobotIo

“写电机”是在 Controller.step()

↓

生成 joint targets ← 这还没有写电机

↓

Safety.apply()

↓

RobotIo.write() ← 这里才真正写电机

↓

Dynamixel

由 RobotIo 执行实际的 Dynamixel 写操作。因为是串口电机，因此实际是调用 Dynamixel 的串口输出。

6.1.2.3.27 Tick 第 20 块：发布 RobotState

6.1.2.3.28 Tick 第 21 块：记录 loop health

6.1.2.4 robotd 的一些主要设计要点

1. 启动后不会自动让机器人站起来。启动后，Duck 处于瘫软状态，为进入 update 准备。

2. 外部控制不是直接控制电机。比如键盘不是直接发指令给电机，而是：
   gamepad
   ↓
   robot.move
   ↓
   IPC
   ↓
   Intents
   ↓
   control loop
   即键盘发出机器人移动命令，通过 IPC（进程间通信）作为 intents 给控制循环（前面讲的控制线程），由控制线程控制电机。

3. Intents 是 IPC 和 control loop 之间的“缓冲层”

intents.rs 是非常重要的文件。里面不是 mutex[3]，而是多个独立的 slot[4]。

每一个 50 Hz tick，control loop 执行：

let snapshot = intents.snapshot();

一次性得到这一 tick 的输入。拿到 Intent 后：

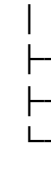
let (gated, deadman) =

safety.gate(snapshot.command, snapshot.twist_age);

Intent



Safety.gate()



如果 gamepad 不再发送：robot.move

最后：

twist_age > deadman timeout

就会被 safety gate 截断。所以 gamepad 断线不会导致：

最后一次 vx=1.0

机器人永远保持 vx=1.0

而会进入 deadman。

---

3 Mutex（互斥锁） 是多线程编程中用于实现线程同步的核心机制，全称为 Mutual Exclusion（互斥），核心作用是保证同一时刻只有一个线程能访问共享资源，彻底避免多线程并发操作引发的数据竞争问题。

核心原理与作用

互斥访问：它就像一把带唯一钥匙的锁，线程在进入访问共享资源的临界区前必须先获取锁，操作完成后释放锁；其他试图获取已被占用锁的线程会被阻塞，直到锁被释放才能继续执行 。

解决数据竞争：当多个线程同时读写同一块共享内存（如全局计数器、公共文件）时，没有同步机制会导致操作结果不可预测，mutex 通过强制序列化访问，保证共享数据操作的逻辑原子性，避免数据损坏 。

严格所有权：互斥锁具有线程相关性，只有持有锁的线程才能释放它，跨线程释放锁会直接触发异常或程序错误 。

不同编程语言和系统中提供了适配不同场景的 mutex 类型：

类型	核心特性	适用场景
普通互斥锁（std::mutex）	基础不可递归，重复加锁直接死锁	常规单重临界区保护，C++ 标准库默认类型
递归互斥锁（recursive_mutex）	允许同一线程多次加锁，需对应次数解锁 递归函数	嵌套调用中需要重复访问同一共享资源的场景
计时互斥锁（timed_mutex）	支持带超时的非阻塞加锁操作	避免线程无限期等待，防止死锁卡死

系统级命名互斥锁	跨进程可见，可同步不同进程中的线程	多进程共享全局资源的场景，如 Windows 系统全局同步

基础使用规范：

1、标准流程遵循「加锁 → 操作共享资源 → 解锁」的模式，直接手动调用 lock()/unlock() 容易因异常、提前返回导致忘记解锁，引发死锁。

2、现代编程实践中优先使用 RAII 风格的自动锁管理类，如 C++ 的 std::lock_guard、Go 的 defer mu.Unlock()，在对象构造时自动加锁，析构时自动解锁，从语法层面避免遗漏解锁操作 。

3、关键注意事项：

避免死锁：多个线程获取多把锁时，必须按全局统一的固定顺序申请锁，禁止循环等待 。

控制锁粒度：只对真正需要保护的临界区代码加锁，不要在持有锁时执行耗时 I/O、网络请求等操作，避免大幅降低程序并发性能 。

禁止在中断上下文使用 mutex：mutex 加锁可能导致线程休眠，中断上下文不允许休眠，否则会引发系统崩溃 。

4 slot 在 Rust 官方文档与标准库实现中，线程相关的 Slot（插槽）并非单一独立概念，而是作为多线程/并发场景下的底层存储单元，分布在通道实现、Tokio 调度器、线程本地存储三大核心场景中，所有定义均来自 Rust 标准库源码与官方公开文档。

通道（Channel）中的消息 Slot

这是 Rust 标准库中定义最明确的 Slot 概念，是 mpsc 通道底层实现的核心存储单元，在标准库源码中直接以 struct Slot<T> 形式定义 。

核心结构：每个 Slot 包含两个字段：

msg: UnsafeCell<MaybeUninit<T>>：用于未初始化状态下安全存储消息数据，避免不必要的内存初始化开销

state: AtomicUsize：原子状态标记位，用三个常量标记 Slot 生命周期：WRITE(1) 表示消息已写入、READ(2) 表示消息已被读取、DESTROY(4) 表示槽位正在销毁回收

无界通道的块式管理：无界通道采用链表块结构，每个 Block<T> 块内置 BLOCK_CAP=31 个 Slot，连续分配多个消息到同一块中，大幅降低内存分配频率、提升缓存命中率

无锁同步逻辑：发送线程通过原子操作标记 WRITE 位，接收线程自旋等待该标记后读取数据，全程无需互斥锁即可完成并发安全的消息传递

Tokio 调度器的 LIFO Slot

Tokio 多线程异步调度器中的专属优化组件，属于 Tokio 官方文档明确标注的调度特性，用于优化消息传递场景下的任务延迟 。

核心作用：每个 worker 线程维护一个独立的专属 Slot，当任务 A 发送消息唤醒任务 B 时，任务 B 不会被推入任务队列尾部，而是直接存入当前线程的这个 Slot 中，下一次调度时优先执行任务 B，大幅降低消息传递的跨线程延迟

特性与配置：该 Slot 中的任务无法被其他工作线程窃取，当任务的 poll 执行时间较长时，可能导致整体吞吐量下降。官方提供 disable_lifo_slot() 配置方法可关闭该优化，将所有任务推入可被窃取的本地队列

4. 真正的 50 Hz tick 这就是核心。
   每 20 ms：ticker.tick().await;
   并且：MissedTickBehavior::Skip
   不是 Burst，也不是 Delay。因此如果某一次执行太慢：
   tick 1
   tick 2
   tick 3 ← missed
   tick 4
   不会：
   tick 3
   tick 3
   tick 3
   补回来。
   也不会因为一次延迟导致整个周期永久漂移。

6.2 ROBOTD 中关于 ONNX 的模型声明、例化、调用

MicroDuck 并不是 robotd 自己直接管理 ONNX Runtime 的全部细节。真正的 ONNX 模型加载、session 创建、shape 校验、inference 都已经抽到了 duck-control::policy，即在 control 循环中；robotd 负责参数解析、策略选择和 50 Hz 控制循环。

最核心的文件：duck-control/src/policy.rs。

6.2.1 定义：定义一个 ONNX 模型对象

文件：`duck-control/src/policy.rs`。定义阶段只是声明“我有一个 ONNX 模型对象”。还没有加载 `.onnx` 文件，也还不能执行。

6.2.2 创建：把 `.onnx` 文件加载成 `Session`

还是在：`duck-control/src/policy.rs`，创建发生在 `Network::open()` 里面。

线程本地存储（TLS）中的数据 Slot

线程本地存储场景下，每个独立线程为 TLS 变量分配的专属存储位置，是 Rust 标准库 std::thread::LocalKey 与第三方 thread_local 库的底层实现细节 。

标准库实现：LocalKey 内部通过间接跳转 thunk 函数，为每个线程动态分配专属 Slot 存储 TLS 数据，保证每个线程访问到完全独立的副本，无需任何同步操作

第三方库实现：thread_local 库通过分层原子指针桶结构，为每个线程分配唯一的 Slot 索引，实现无锁的 TLS 数据访问，性能优于系统原生 pthread TLS 接口

生命周期规则：每个线程首次访问 TLS 变量时，该线程对应的 Slot 会自动执行惰性初始化；线程退出时，Slot 内的数据自动执行 Drop 析构

6.2.3 `robotd` 创建它

`Policy::load()` 内部：

```
Policy::load()
      |
      ├──── Network::open(walk.onnx)
      ├──── Network::open(stand.onnx)
      ├──── Network::open(...)
      └──── ...
```

```
robotd
   |
   |  Policy::load()
  ▼
```

```
Policy
   |
   └──── Network
           |
           └──── ort::Session
```

对于 walking：

```
alpha_walking.onnx
          ↓
Network
          ↓
Session
```

6.2.4 执行：调用 ONNX

真正执行发生在：

`duck-control/src/policy.rs`

`Network::run()`。

核心可以简化成：

let outputs = self.session.run(inputs)?;

这就是最关键的一句。

也就是说：

```text

observation

    |

    ▼

ONNX input tensor

    |

    ▼

self.session.run(...)

    |

    ▼

ONNX Runtime

    |

    ▼

[1, 14]```

### 6.2.5 `robotd` 是怎么调用它的？

在控制循环中，`robotd` 最终会走到：

self.policy.infer(&observation, net)

然后：

Controller::step()

        |

        ▼

Policy::infer()

        |

        ▼

Network::run()
```

|

▼

session.run()

|

▼

ONNX

|

▼

14 actions

所以你只需要记住这条调用链：

robotd

|

▼

Policy::infer()

|

▼

Network::run()

|

▼

Session::run()

|

▼

alpha_walking.onnx

6.2.6 三个动作总结

① 定义

告诉 Rust：

```
struct Network {
    session: Session,
```

}

意思：一个 Network 里面有一个 ONNX Runtime Session。

② 创建

```
let session = Session::builder()
    .commit_from_file(path)?;
```

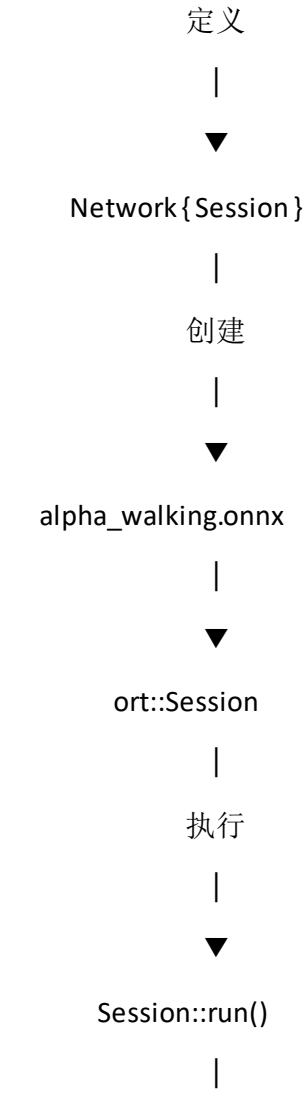
意思：把 `alpha_walking.onnx` 加载进 ONNX Runtime，创建 `Session`。

③ 执行

```
let outputs = self.session.run(inputs)?;
```

意思：把 61 维 observation 喂给 ONNX，执行神经网络，得到 14 维 action。

最简单的一张图



▼

[1, 61] → [1, 14]

定义 声明对象长什么样。

创建 = 加载 ONNX 文件，得到 `Session`。**

执行 =`Session::run()`，输入 observation，输出 action。**

这三个层次分清楚后，再看 `robotd` 的其它代码就不会乱了。

6.3 从 ONNX 文件到程序输出：完整流程

这段讲述一般的 ONNX 概念和工作流程。

6.3.1 先理解：ONNX 文件本身是什么

ONNX（Open Neural Network Exchange）文件本质上是一个“静态计算图描述”——它只描述"数据怎么流动、经过哪些算子"，不包含执行引擎本身。所以一个“.onnx”文件在程序里是“跑不起来的”，而必须：

ONNX 模型文件 → 推理引擎（Runtime） → 你的程序拿到输出

这正是 MicroDuck 架构里分层的原因：策略训练在 `microduck_rl`（PyTorch + PPO）里完成（后面的章节讲述），导出成 ONNX 后，MicroDuck 机器人上的`robotd`守护进程用 ONNX Runtime 加载它以 50Hz 实时推理——训练框架和推理引擎完全解耦。MicroDuck 控制循环就是`状态→观测→神经网络(ONNX)→动作→物理→新状态`的闭环。

6.3.2 完整过程（四步）

第 1 步：确认"模型契约"（最容易被忽略的一步）

拿到 ONNX 文件第一件事不是写代码，而是搞清楚它的输入输出契约——形状、数据类型、语义。以 MicroDuck 的行走策略为例：

-输入：`obs`[5]，float32，shape[6]`[1, 61]`

-输出：`actions`，float32，shape`[1, 14]`（对应 14 个舵机的动作）

-控制频率：50Hz（每 20ms 推理一次）

-重要细节：观测归一化器已经烘焙进了 ONNX 图内部，调用方不要再自己归一化。

查看方法（任选其一）：

```python
import onnx
model = onnx.load("policy.onnx")
print(onnx.helper.printable_graph(model.graph))
```

或在线工具 netron.app 可视化打开。

第 2 步：安装推理引擎

最常见的选择是 ONNX Runtime（微软维护，支持 Python/C++/C#/Rust/Java，可在 CPU 上零依赖运行；RK3566/RDK X5 这类 ARM64 板上也有对应预编译包：

```bash
pip install onnxruntime
```

---

5 obs=observation（观测值）的缩写。这是强化学习、机器人、自动驾驶里*最常见的*变量名之一。ONNX 导出的强化学习策略模型很多都把输入节点命名成 obs。举个四足机器人例子，Unitree、Isaac Lab、Isaac Gym 里很常见：

obs =[身体角速度 3 维

重力方向 3 维

12 个关节位置

12 个关节速度

上一步动作

目标速度]

6 在 ONNX、Tensor、深度学习里，Shape（形状） 是最最重要的概念之一。可以简单理解为：

Shape 描述了一个张量（Tensor）每个维度有多大，类似于矩阵的维数。如：input: float32[1,3,224,224]，这里面 shape 就是[1,3,224,224]。`obs`[ ]，float32，shape[ ]`[1, 61]`的意思就是：obs 变量是 float32 类型的 61 维向量。

如果有 NVIDIA GPU：pip install onnxruntime-gpu

```

备选引擎（按场景）：TensorRT（英伟达加速）、RKNN（瑞芯微 NPU）、ONNX Runtime Web（浏览器，MicroDuck 官方沙盒就是这么做的）。

第 3 步：加载模型，跑一次推理

Python 完整最小示例（对应 MicroDuck 契约）：

```python
import numpy as np
import onnxruntime as ort

①创建推理会话（Session 是资源句柄，只创建一次，别放进循环里）
sess = ort.InferenceSession("policy.onnx", providers=["CPUExecutionProvider"])
② 确认输入输出（与模型契约核对）
inp = sess.get_inputs()[0]
print(inp.name, inp.shape, inp.type)    # 例如: obs [1, 61] tensor(float)
③ 准备输入：严格按契约构造 float32 数组
obs = np.zeros((1, 61), dtype=np.float32)    # 实际应从 IMU/舵机读取观测
④ 推理：传入 {输入名: 数组}，拿到输出列表
actions = sess.run(None, {inp.name: obs})[0]
print(actions.shape)    # (1, 14)
```

C++ 等价写法（真机 `robotd`的 Rust 逻辑本质相同）：

```cpp
#include <onnxruntime_cxx_api.h>
Ort::Env env(ORT_LOGGING_LEVEL_WARNING, "demo");
Ort::Session session(env, L"policy.onnx", Ort::SessionOptions{});
std::vector<int64_t> dims{1, 61};
std::vector<float> obs(61, 0.0f);
```
```

```
auto mem = Ort::MemoryInfo::CreateCpu(OrtArenaAllocator, OrtMemTypeDefault);
Ort::Value input = Ort::Value::CreateTensor<float>(mem, obs.data(), obs.size(), dims.data(), dims.size());
auto outputs = session.Run(Ort::RunOptions{nullptr},
                           session.GetInputNames().data(), &input, 1,
                           session.GetOutputNames().data(), 1);
float* actions = outputs[0].GetTensorMutableData<float>();
```

Rust 中可用 `ort` crate；这也是 MicroDuck 官方栈在用的技术路线。

第 4 步：嵌入控制循环

单次 `sess.run` 拿到的是原始张量；在机器人程序里，你要把它放进实时循环：

```python
import time
RATE_HZ = 50                                # 与训练时的控制频率严格对齐！
dt = 1.0 / RATE_HZ

while True:
    t0 = time.time()
    obs = read_sensors()                    # ① 从 IMU/舵机编码器组 61 维观测
    actions = sess.run(None, {"obs": obs})[0]   # ② ONNX 推理 → 14 维动作
    send_to_servos(actions[0])              # ③ 动作下发到舵机总线
    time.sleep(max(0, dt - (time.time() - t0)))   # ④ 补足到 50Hz
```

工程要点：

1.频率对齐：训练时 50Hz 训出的策略，部署时也必须 50Hz 调用，否则观测的时间语义错位，策略会失衡。

2.预处理/后处理与模型约定一致：比如 MicroDuck 的归一化已烘焙进图，就别重复做；如果导出时没烘焙，就要在代码里手动做。官方导出器 `scripts/export.py` 会自动烘焙归一化器，这是正确行为所必需的。

3.会话只建一次，循环里只调用 `run()`；每次新建 Session 的开销在实时控制中不可接受。

7 MICRODUCK 的训练

地址：https://github.com/pollen-robotics/microduck_rl

入手点：

- README.md：掌握「仿真→训练→部署」全流程（基于 MuJoCo Warp 仿真、PPO 算法）；
- src/agents/：强化学习智能体实现（ reward 设计、策略网络结构）；
- scripts/export.py：模型导出为 ONNX 格式的脚本（连接仿真与硬件的关键）。

核心闭环：Sim2Real 链路（架构设计核心）

这是 Microduck 软件架构的灵魂，也是开源的核心价值：

1. 仿真训练：在 MuJoCo Warp 中搭建机器人模型，通过 PPO 算法训练行为策略（如行走、跌倒恢复）；
2. 模型导出：将训练好的策略导出为 ONNX 格式（microduck_rl/scripts/export.py）；
3. 硬件部署：机器人通过 ONNX Runtime 加载模型，在 50Hz 控制环中执行；
4. 迭代优化：通过传感器数据观察硬件与仿真的差异，调整仿真参数（如摩擦、延迟）后重新训练。

第 1 章 文档说明与训练流程总览

1.1 文档目标与使用方式

本文档的目标是：只阅读本文档，即可按照叙述完成 MicroDuck 机器人强化学习训练的全流程，包括环境准备、任务选择、冒烟测试、正式训练、训练监控、恢复与热启动、结果评估、ONNX 导出、发布到 Hugging Face 以及真机安装。每一步都给出可直接复制的命令、参数说明与预期结果。

建议按章节顺序执行。前四章（环境、任务、冒烟）是正式训练前的必经步骤，不可跳过；第五章是训练主体；第六至七章是训练完成后的评估、导出与发布；第八、九章用于遇到问题时的排查与命令速查。

1.2 训练全流程总览

表 1 训练全流程总览

步骤	内容	关键命令
1	环境准备	git clone && uv sync
2	选择任务	uv run list-envs
3	冒烟测试	uv run train <TASK_ID> --env.scene.num-envs 64 --agent.max_iterations 5
4	正式训练	uv run train Mjlab-Velocity-Flat-MicroDuck --env.scene.num-envs 4096
5	监控与恢复	wandb 页面 / logs 目录 / resume
6	结果评估	uv run play / scripts/infer_policy.py
7	导出 ONNX	uv run scripts/export.py <TASK_ID> --wandb-run-path <...>
8	发布与部署	uv run publish / robotctl policy

1.3 关键概念与术语

MicroDuck 是一台约 800 克、约 25 厘米高的双足机器人，装有 14 个 Dynamixel XL330 舵机。本仓库为其提供强化学习训练环境，训练得到的策略导出为 ONNX 后在真实机器人上由运行时加载执行。

mjlab：训练框架，基于 MuJoCo Warp（GPU 物理仿真）与 PPO（rsl_rl 实现）。训练控制频率为 50 Hz。

BAM 执行器模型：sim2real 的关键。用电压控制律、反电动势、库仑/Stribeck/负载相关摩擦对 XL330 建模，并对电池电压、负载电压跌落、指令延迟与摩擦大小做逐环境域随机化（FrictionDRBamActuator）。训练与部署使用同一执行器模型。

观测/动作契约：所有策略共享 61 维 actor 观测（48 维本体感知 + 13 维指令块［twist(3)、head_pose(4)、body_pose(6)］）与 14 维动作；ONNX 导出图为 [1,61]→[1,14]。不使用某个指令槽的环境对该槽零填充，而不是删除该槽。

观测归一化：训练时使用经验归一化（EmpiricalNormalization），导出 ONNX 时归一化参数烘焙进计算图，因此部署必须使用 scripts/export.py 导出的 ONNX，禁止手转 checkpoint。

Backlash 变体：每个主任务都有带 ±1°（共 2°）齿隙模型的孪生任务，更接近真实舵机传动。

域随机化开关：每个任务配置文件的顶部以 ENABLE_* 布尔变量列出，可自行开关。

1.4 仓库与相关项目

仓库地址：https://github.com/pollen-robotics/microduck_rl，许可证为 Apache-2.0（3D 模型文件为 CC BY-SA-NC）。相关项目包括：microduck（机器人项目主页，含运行导出策略的机载运行时）、mjlab（训练框架）、BAM（执行器模型库，Rhoban 出品）。

第 2 章 环境准备

2.1 硬件要求

训练必须使用 CUDA GPU，因为训练经 MuJoCo Warp 在 GPU 上运行物理仿真。标准训练配置为 4096 个并行环境（--env.scene.num-envs 4096）。若本机没有 GPU，可将训练提交到 Hugging Face Jobs 云端执行（见 5.7 节），无需本地 GPU。

在 ARM 盒子（如 DGX Spark / GB10、Jetson）上，首次 uv sync 需要拉取约 2 GB 的 CUDA wheels，uv 默认的 30 秒 HTTP 超时可能中断下载，此时需要先设置 UV_HTTP_TIMEOUT=600 再执行（见 2.4 节）。

注意：4096 个并行环境对显卡显存要求较高，官方建议使用显存较大的 NVIDIA 显卡（如 RTX 40 系列）。显存 6 GB 左右的显卡（如 GTX 1660 Ti）请先以 1024～2048 个环境运行冒烟与训练，确认不出现显存不足（OOM）后再逐步增加；冒烟测试的 64 个环境对显存要求很低，CPU 也可运行。

2.2 软件要求

需要安装以下软件：

git（用于克隆仓库）。

uv（Python 包与项目管理器，仓库所有命令均通过 uv run 执行）。

Python 3.12（pyproject.toml 锁定 Python 版本为 >=3.12,<3.13；uv 会自动下载并使用匹配版本，无需手动安装）。

uv 的安装方式：macOS/Linux 执行 curl -LsSf https://astral.sh/uv/install.sh | sh；Windows 在 PowerShell 中执行 powershell -c "irm https://astral.sh/uv/install.ps1 | iex"。安装完成后可用 uv --version 验证。

文中命令在 Linux/macOS 的 Bash 中验证。Windows 用户建议在 PowerShell（命令写法一致）或 WSL2 中执行；需要 GPU 训练时，请确保驱动与 CUDA 环境可被 PyTorch 使用。

在 Windows 上通过 PowerShell 调用 wsl 命令时，命令行中的双引号、$HOME/$PATH 变量与 && 等符号可能被 Windows 层改写，导致命令失效或输出被吞

（退出码为 0 但无任何输出）。最稳妥的做法是直接打开 WSL 终端（Ubuntu 窗口）逐条执行本文命令，或将多条命令写入 shell 脚本后一次性执行。

2.3 克隆仓库

打开终端，执行以下命令克隆仓库并进入目录：

```
git clone https://github.com/pollen-robotics/microduck_rl
cd microduck_rl
```

仓库包含训练环境源码（src/mjlab_microduck/）、机器人 MJCF 模型（含 3D 打印件）、脚本（scripts/）、文档（docs/、README.md、AGENTS.md）与测试（tests/）。

2.4 安装依赖

在仓库根目录（后续 uv 的命令都要在仓库根目录中运行）执行 uv sync，安装全部依赖（包括 PyTorch、mjlab、MuJoCo Warp、BAM 等）：

```
uv sync
```

ARM 盒子（DGX Spark / GB10、Jetson）首次同步请先设置超时：

```
UV_HTTP_TIMEOUT=600 uv sync
```

依赖版本已由 pyproject.toml 锁定。特别说明：在 linux-aarch64 上，PyPI 的 torch wheel 是 CPU 版，pyproject.toml 中 [tool.uv.sources] 已为 aarch64 将 torch 路由到 cu129 索引（x86_64 与 HF Jobs 仍使用 PyPI）。请勿删除 torch 直接依赖或把 == 引脚改为 >=，否则路由失效会装回 CPU 版（详见第 8 章）。

依赖下载总量较大（torch 与 CUDA 相关库合计约 6～8 GB），请预留至少 20 GB 磁盘空间。若 uv sync 中途失败（常见于网络受限环境，错误信息形如 Request failed ... tunnel error / tcp connect error / deadline has elapsed，且仓库目录下未生成 .venv 目录），说明 PyPI 下载通道不稳定，按以下顺序处理：

（1）配置国内镜像（推荐）：创建用户级配置文件 ~/.config/uv/uv.toml，写入以下内容后重新执行 uv sync：

```
[[index]]
url = "https://pypi.tuna.tsinghua.edu.cn/simple"
default = true
```

（2）或单次执行 UV_DEFAULT_INDEX=https://pypi.tuna.tsinghua.edu.cn/simple uv sync（等效于上面的配置，仅对本次命令生效）。

（3）若系统设置了代理环境变量（http_proxy/https_proxy 等）且镜像直连仍失败，可临时执行 unset http_proxy https_proxy HTTP_PROXY HTTPS_PROXY ALL_PROXY 后再运行 uv sync。

（4）uv sync 失败后直接重跑即可续传：已下载的包缓存在 ~/.cache/uv/ 目录，反复失败时每次失败的包通常不同，可循环重试直至成功。

使用镜像执行 uv sync 会把仓库内 uv.lock 的包下载地址改写为镜像地址（git status 可见 uv.lock 被修改），属正常现象，不影响后续训练；保留该修改可让之后的 uv run、uv sync 直接从镜像下载，速度更快更稳定。

2.5 验证安装

运行以下命令验证环境可用：

```
uv run list-envs
```

预期输出：当前代码注册的全部训练环境（任务）清单。若正常列出任务（见第 3 章），说明环境与依赖安装成功。

再检查 GPU 是否可用（无 GPU 时该检查为 False 属正常，可改用 5.7 节 HF Jobs 方案）：

```
uv run python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
```

也可用 uv run python -c "import mjlab_microduck" 确认项目包可导入。

判断 uv sync 是否真正成功：仓库目录下应存在 .venv 目录（可用 ls .venv/bin 查看可执行文件）。若 .venv 不存在或为空，说明依赖未装全，按 2.4 节方法重试。首次运行 uv run 可能再次联网核对依赖索引，请保持网络与镜像配置一致。

2.6 （可选）账号登录

训练监控与发布需要以下账号（均为可选，本地训练不登录也能进行；但发布与 HF Jobs 必须）：

（1）wandb（Weights & Biases）——训练监控平台。是什么：机器学习实验跟踪工具，注册地址 https://wandb.ai （点击 Sign Up 注册免费账号），注册后执行 wandb login 登录（首次会提示粘贴 API key）。有什么用：训练时把平均奖励、回合长度、各惩罚项等指标实时同步到云端面板，可绘制曲线、多 run 对比、远程查看训练进度（判断方法见 5.3 节）。什么时候必需：不登录也能本地训练（指标同时写入本地 logs/ 目录）；但要在浏览器查看曲线，以及用 --wandb-run-path 做恢复训练、回放评估（play）、ONNX 导出（export.py）时，需要对应的 wandb run 记录，建议登录。是否收费：个人与学术使用免费（官方 Free 计划 0 美元/月，个人项目不限量；企业商业用途才按席位收费），登录即可使用，无需付费。

（2）Hugging Face——模型托管与云端训练平台。是什么：AI 模型与数据集托管平台，注册地址 https://huggingface.co （点击 Sign Up 注册免费账号），注册后执行 hf auth login 登录，或设置环境变量 HF_TOKEN。有什么用：一是本机无 GPU 时，把训练提交到 Hugging Face Jobs 云端 GPU 执行（训练命令加 --hf-jobs，见 5.7 节），检查点自动上传回你的账号；二是训练完成后用 uv run publish 把 ONNX 策略发布到 HF Hub，真机用 robotctl policy 一键安装（见第 7 章）。什么时候必需：仅发布与 HF Jobs 两个场景必须登录，纯本地训练不需要。是否收费：注册账号、托管公开模型与数据集均免费

（Pro 订阅为可选增强）；但 HF Jobs 云端训练按 GPU 小时计费，以官方 hf.co/pricing 为准，常用档位参考：l4x1（L4，24 GB）约 0.8 美元/小时、a10g-large（A10G，24 GB）约 1.5 美元/小时、a100-large（A100，80 GB）约 2.5 美元/小时；本地训练不产生任何费用。

第 3 章 选择训练任务

3.1 查看任务注册表

以 uv run list-envs 的输出为活体注册表（代码更新后任务会变化）。下文表格来自仓库 README，供规划参考；实际可训练任务以 list-envs 为准。

3.2 任务清单

表 2 任务清单（来自 README Tasks）

任务 ID	地形	说明
Mjlab-Velocity-{Flat,Rough}-MicroDuck	平地/崎岖	主任务：速度指令 + 头部姿态指令行走
Mjlab-VelStand-{Flat,Rough}-MicroDuck	平地/崎岖	单一策略内完成行走 + 跌倒恢复
Mjlab-StandUp-{Flat,Rough}-MicroDuck	平地/崎岖	从面朝下/面朝上/坐姿起身，保持站立并支持身体姿态控制
Mjlab-SitStand-{Flat,Rough}-MicroDuck	平地/崎岖	受控的坐↔站切换（动作轻柔），头部可指挥
Mjlab-GroundPick-{Flat,Rough}-MicroDuck	平地/崎岖	下蹲用嘴尖触地，然后回到站立
Mjlab-BallKick-Flat-MicroDuck	平地	将 70 mm/15 g 球向前踢出（actor 对球不可见）
Mjlab-Roulade-Flat-MicroDuck	平地	前滚翻，落地回到双脚
Mjlab-Velocity-Flat-MicroDuck-Rollers	平地	轮滑速度跟踪（脚下为被动轮）
Mjlab-Velocity-Swizzle-MicroDuck	平地	经典对称 swizzle 滑行
Mjlab-RollerCrouch-Flat-MicroDuck	平地	轮滑滑行中下蹲
Mjlab-RollerSlope-Flat-MicroDuck	坡道	轮滑下坡滑行
Mjlab-RollerStandUp-Flat-MicroDuck	平地	从地面起身站上轮滑
Mjlab-Spin-Flat-MicroDuck	平地	轮滑原地快速旋转

初学者建议从主任务 Mjlab-Velocity-Flat-MicroDuck 开始：README 的快速开始即以此为例，训练难度与时长适中，且可作为其他任务的起点。

3.3 任务命名规则与 Backlash 变体

任务 ID 形如 Mjlab-<任务>-<地形>-MicroDuck，如 Mjlab-Velocity-Flat-MicroDuck、Mjlab-StandUp-Rough-MicroDuck。

每个主任务都有一个 Backlash 孪生任务：在任务 ID 中、MicroDuck 之前插入 -Backlash，例如 Mjlab-Velocity-Flat-Backlash-MicroDuck。Backlash 模型在 14 个舵机关节上串联 ±1° 齿隙（共 2°）。齿隙被正确建模：每个舵机带一个未驱动的 passive_<关节>_backlash 铰链，真实编码器位于齿隙输出侧，因此固件 PD 仿真与 joint_pos/joint_vel 观测都“透过”齿隙读取。观测与动作维度不变，ONNX 导出与运行时无需改动。Backlash 策略通常更贴近真实舵机表现，若追求最强 sim2real，可训练 Backlash 变体。

3.4 机器人与观测/动作契约

机器人模型位于 src/mjlab_microduck/robot/microduck/，由 Onshape 经 onshape-to-robot 导出。不同任务使用不同碰撞模型，见下表。

表 3 机器人模型与用途

XML 模型	用途
robot_walk.xml	Velocity（剥离躯干/头部碰撞——摔倒代价低）
robot_groundcontact.xml	VelStand、StandUp、SitStand、GroundPick、BallKick、Roulade（策划触地碰撞集合，身体可真实贴地）
robot_groundcontact_rollers.xml	轮滑任务（被动轮）
robot_allcollisions.xml	全碰撞模型（每个部件都有碰撞几何体），目前无任务使用
robot_*_backlash.xml	Backlash 变体（由 add_backlash.py 生成）

关节布局（14 个舵机）：0–4 左腿（hip_yaw、hip_roll、hip_pitch、knee、ankle），5–8 颈/头（neck_pitch、head_pitch、head_yaw、head_roll），9–13 右腿。未驱动关节统一以 passive_ 前缀命名（如轮滑被动轮、齿隙铰链）；执行器、关节观测与姿态奖励通过正则 ^(?!passive_).* 只选择舵机关节。

观测契约（所有策略共享）：61 维 actor 观测 = 48 维本体感知 + 13 维指令块［twist(3)、head_pose(4)、body_pose(6)］，按此顺序排列。不使用某指令槽的环境将该槽零填充（保留观测项、采样极小范围），绝不删除槽位。动作维度为 14。

第 4 章 冒烟测试

4.1 为什么必须先冒烟

AGENTS.md 明确要求：任何长训练开始前必须执行冒烟测试。64 个环境、5 次迭代的冒烟测试只需几分钟，就能拦截约 95% 的配置错误（环境构建、观测维度、模型加

载、BAM 执行器、渲染等）。跳过冒烟直接跑长训练，错误会在数小时后才暴露，代价极高。

4.2 执行冒烟测试

以主任务为例，执行：

uv run train Mjlab-Velocity-Flat-MicroDuck --env.scene.num-envs 64 --agent.max_iterations 5

预期结果：环境成功构建；日志显示观测维度 61、动作维度 14；训练正常推进 5 次迭代并正常结束；无 NaN；logs/ 目录下出现本次运行的记录。命令中的 <TASK_ID> 可替换为任意 list-envs 中的任务 ID。

4.3 冒烟测试失败排查

若冒烟失败，常见原因与处理：

任务名错误或未注册：运行 uv run list-envs 核对任务 ID 拼写与当前注册表。

CUDA/环境错误：确认 GPU 驱动、CUDA 与 torch 的 GPU 版（见 2.5 节检查）。

观测/动作维度不匹配：检查是否误删了指令槽或 height_scan 观测项（velocity 配置已删除 height_scan，其他配置应保持一致做法）。

出现 NaN：多为奖励惩罚项符号错误（见 5.3 节与第 8 章）。

第 5 章 正式训练

5.1 启动训练

以主任务为例，标准训练命令（4096 个并行环境）：

uv run train Mjlab-Velocity-Flat-MicroDuck --env.scene.num-envs 4096

README 指出：在 4096 个环境下，约 1–2 小时即可得到可用的步态（取决于 GPU 性能）。训练过程中每迭代打印奖励指标并同步到 wandb；检查点按固定间隔保存到本地 logs 目录（见 5.3 节）。

5.2 训练参数说明

表 4 常用训练参数

参数/项目	说明
--env.scene.num-envs	并行环境数。标准配置 4096；显存受限可调小（如 2048/1024）；冒烟测试用 64。
--agent.max_iterations	最大迭代数，默认 50000，足够覆盖常见任务预算（见 5.4 节）。
--agent.run-name	本次运行的名称（用于区分多次运行，如 resume）。默认与 experiment_name 相同。

参数/项目	说明
--agent.load-checkpoint	要加载的检查点文件名，形如 model_29999.pt，配合 --agent.resume 使用。
--agent.resume	设为 True 时从指定检查点恢复训练（同时恢复权重、观测归一化器与优化器状态）。
--wandb-run-path	wandb run 路径，格式为 entity/project/run_id。恢复训练或基于某 run 评估/导出时使用。
--wandb-checkpoint-name	wandb run 内指定检查点文件名（与 --wandb-run-path 配合）。
--hf-jobs	追加到任意 train 命令后，将训练提交到 Hugging Face Jobs 云端执行（见 5.7 节）。

训练超参与域随机化配置位于任务配置文件 src/mjlab_microduck/tasks/microduck_velocity_env_cfg.py：文件顶部为 ENABLE_* 域随机化开关；文件末尾 MicroduckRlCfg 为 PPO 超参。主行走配方的默认值（velocity 任务）如下：actor/critic 网络为 (512,256,128) 隐藏层、elu 激活、启用观测归一化；高斯分布初始标准差 1.0；PPO 学习率 1e-3（自适应调度）、value_loss_coef 1.0、clip_param 0.2、entropy_coef 0.01、每轮 5 个学习 epoch、4 个 mini-batch、gamma 0.99、lam 0.95、desired_kl 0.01、max_grad_norm 1.0；save_interval 250（每 250 迭代存一次检查点）、num_steps_per_env 24、max_iterations 50000。wandb 项目名为 mjlab_microduck，experiment_name 为 velocity（即日志目录名）。

5.3 训练监控与结果判断

训练指标实时同步到 wandb 项目 mjlab_microduck（run 名称形如 <时间戳>_velocity）。本地日志位于 logs/<experiment_name>/ 目录（velocity 任务即 logs/velocity/），检查点保存为 model_<迭代号>.pt，例如 model_250.pt、model_500.pt，每隔 save_interval（默认 250）迭代生成一个。

判断训练是否正常，逐迭代观察以下几点：

平均奖励（mean reward）持续上升。

回合长度（episode length）符合任务预期（如行走任务不应过早结束）。

每个惩罚项满足 Episode_Reward/<惩罚名> ≤ 0（惩罚项以正权重乘自否定值，若权重为负会造成双重否定、把违规变成奖励，训练必出 NaN 或畸形行为）。

主任务项真正增长：总奖励可能只靠正则项上升而技巧从未发生，需确认任务核心项在增长。

注意：Episode_Reward/<term> 记录的是加权值——权重为 0 的项无论行为如何恒为 0，解读时要结合权重调度。

训练结束或提前停止后，在 logs/velocity/ 目录下选择检查点。经验：简单技巧任务的可用检查点常出现在 1000 次迭代附近；步态类任务在 4000–6000 次迭代附近（见 5.4 节）。

5.4 训练时长预算

AGENTS.md 给出的预算（4096 环境）：

简单 episodic 技巧（如 Roulade、SitStand）：约 1000 次迭代。

步态与课程化恢复类（如 Velocity、VelStand）：4000–6000 次迭代。

max_iterations 默认 50000，覆盖上述预算绰绰有余。达到目标可提前结束（Ctrl+C 终止，已保存的检查点不受影响）；也可用 --agent.max_iterations 预先限制迭代数。

5.5 从检查点恢复训练

训练中断或想继续训练时，用 --agent.resume 从检查点恢复（同时恢复权重、观测归一化器与优化器状态）：

```
uv run train Mjlab-Velocity-Flat-MicroDuck --env.scene.num-envs 4096 \
    --agent.run-name resume --agent.load-checkpoint model_29999.pt --agent.resume True
```

检查点文件位于 logs/velocity/ 目录；若从 wandb 恢复，使用 --wandb-run-path <entity/project/run_id> 与 --wandb-checkpoint-name model_XXXX.pt。

5.6 跨任务热启动（Warm Start）

warm start 与 resume 不同：mjlab 的 runner 会把 common_step_counter 存入检查点并在加载时恢复（连同迭代号），因此直接加载别的任务的检查点，会让所有基于步数的课程在第 1 次迭代就跳到最终阶段。若想用另一个任务的检查点初始化新任务、并让新任务的课程从阶段 0 重新跑，必须在命令前加环境变量 MICRODUCK_WARM_START=1：

```
MICRODUCK_WARM_START=1 uv run train Mjlab-VelStand-Rough-Backlash-MicroDuck --env.scene.num-envs 4096 \
    --agent.resume True --wandb-run-path pollen-robotics/mjlab_microduck/fhathosb \
    --wandb-checkpoint-name model_3750.pt --agent.max_iterations 1500
```

warm start 会保留权重、观测归一化器与优化器，同时把 common_step_counter 与迭代号归零，使课程重新从阶段 0 运行。同时应在目标配置中把源任务的继承课程折叠到最终阶段（可参考 velstand 任务中的 _collapse_curricula_to_final 实现），因为加载的策略正是在这些条件下训练的。

注意：只应在观测归一化器兼容的任务之间热启动。例如站立专家的 twist 标准差为 0.005，绝不能用它来初始化行走环境。

5.7 无 GPU 方案：Hugging Face Jobs

本机无 GPU 时，在任何 train 命令后追加 --hf-jobs，即可把训练提交到 Hugging Face Jobs 云端执行（详见仓库 scripts/hf/README.md）。

一次性准备（只需一次）：

执行 hf auth login 登录 Hugging Face（或设置环境变量 HF_TOKEN）。

执行 wandb login 登录 wandb（HF Jobs 训练同样记录到 wandb 项目 mjlab_microduck）。

提交训练（以主任务为例）：

uv run train Mjlab-Velocity-Flat-MicroDuck --env.scene.num-envs 4096 --hf-jobs

表 5 HF Jobs 常用参数

参数/项目	说明
--namespace	HF 命名空间（你的用户名或组织名）。不传时交互选择，自动化场景建议显式传入。
--flavor	硬件规格：l4x1（默认）、a10g-large、a100-large。
--timeout	任务最大时长，默认 12h。
--detach	提交后不跟随日志输出，立即返回。
--dry-run	只打印提交计划，不真正提交。
--run-name	本次云端运行的名称（决定检查点仓库名）。
--no-uv-cache	不使用共享 uv 缓存桶（默认使用 <namespace>/mjlab-uv-cache）。
--no-wandb	云端训练不写 wandb。

机制说明：提交时本地源码被打包成 tarball 上传到私有 dataset 仓库，任务在云端容器内执行 uv sync（保持 pyproject.toml 一致，确保远程可复现）；训练过程中的检查点由 uploader.py 每 60 秒上传一次到 HF 模型仓库 <namespace>/<run-name>，可直接在 Hugging Face 页面浏览和下载 model_XXXX.pt 文件。提交成功后终端会打印 job id、日志地址与检查点仓库链接。

任务管理（在 Python 中）：可用 huggingface_hub 的 HfApi 管理任务，例如 HfApi().list_jobs(namespace=...)、fetch_job_logs(job_id=..., namespace=...) 查看日志、cancel_job(job_id=..., namespace=...) 取消任务。

第 6 章 评估训练结果

6.1 查看器回放（uv run play）

用查看器回放某次 wandb run 的策略（加载检查点后在 MuJoCo 仿真中观看）：

uv run play Mjlab-Velocity-Flat-MicroDuck --wandb-run-path <entity/project/run_id>

将 <entity/project/run_id> 替换为 wandb 中的实体/项目/运行 ID。

6.2 部署演练（infer_policy.py）

scripts/infer_policy.py 在 CPU MuJoCo 中加载导出的 ONNX 策略，用键盘实时驱动，模拟真实运行时的策略热切换（行走/站立/技巧任意切换）。这是部署到真机前最重要的演练工具。一次可加载多个策略：

```
uv run scripts/infer_policy.py --walking walk.onnx --standing stand.onnx \
    --sitstand sitstand.onnx --roulade roulade.onnx --new-cmd-obs
```

表 6　infer_policy.py 键盘控制

按键	动作
W/S	前进/后退线速度
A/D	左右横移线速度
Q/E	原地转向角速度
G	触发地面拾取策略
Y	坐⇔站切换（sitstand 策略）
R	触发前滚翻策略
K/L	触发左脚/右脚踢球策略

表 7　infer_policy.py 常用参数

参数/项目	说明
--walking/--standing/--sitstand/--ground-pick/--kick-left/--kick-right/--roulade/--slope	指定各策略对应的 ONNX 文件路径；至少提供 walking、standing 或 sitstand 之一。
--new-cmd-obs	使用统一 13 维指令观测布局（61 维）。sitstand、kick、roulade 策略必须加此参数。
--debug	逐帧打印观测与动作，用于对照。
--save-csv	把观测与动作保存到 CSV 文件，用于 sim2real 对比。
--record	开启录制：Ctrl+C 时把观测保存为 pickle 文件。
--no-bam	回退使用 XML 位置执行器（非训练所用执行器，仅作对照，一般不用于评估）。
--vin	BAM 电池电压（默认 7.4 V）。训练时逐环境采样电压，此处钉到一个值。
--vin-drop-gain	负载相关电压跌落增益（默认 0.1 V/Nm），对应训练 DR 范围。
--kp-fw	BAM 固件 P 增益（默认同训练值）。
--roller	使用轮滑机器人模型（robot_walk_rollers.xml）。
--scene	指定场景 XML 路径，覆盖默认场景选择。

重要：infer_policy.py 的舵机使用与训练一致的 BAM M6 XL330 模型（电压控制 + 负载相关摩擦，经 bam.mujoco.MujocoController）；--vin、--vin-drop-gain、--kp-fw 用于把训练时的域随机化范围钉到一个确定值，以便与实机对照。

6.3 回归测试

仓库自带配置不变性与奖励函数回归测试（CPU 即可运行，无需 GPU），锁定关节索引映射、奖励符号约定与 NaN 防护。修改配置后建议运行：

```
uv run --with pytest pytest tests/
```

第 7 章 导出 ONNX 与发布

7.1 导出 ONNX

将训练好的检查点导出为部署用 ONNX：

```
uv run scripts/export.py Mjlab-Velocity-Flat-MicroDuck --wandb-run-path <...>
```

可选参数 --checkpoint N 指定 run 内的检查点迭代号（如 3000）；不指定时使用默认选择。导出产物为 output.onnx（可指定输出路径），计算图形状为 [1,61]→[1,14]。

关键：导出器会把观测归一化器（EmpiricalNormalization）烘焙进 ONNX 图。部署必须使用 scripts/export.py 导出的 ONNX——手转 checkpoint 的模型在运行时看到的是未归一化观测，行为会异常。

7.2 验证导出文件

用 infer_policy.py 加载导出的 ONNX 确认可正常推理：

```
uv run scripts/infer_policy.py --walking output.onnx
```

确认策略能响应键盘指令、动作合理后，再进入发布步骤。

7.3 发布到 Hugging Face Hub

uv run publish 把策略以机器人守护进程可加载的形式发布到 Hugging Face Hub：一个带观测归一化的 policy.onnx、一个遵循 schema 2 的 manifest.json，以及一个说明运行方式的 README。任何拥有 microduck 的人都能一键安装，无需发布新的守护进程版本。

从 wandb run 导出并发布（经唯一安全路径导出后上传）：

```
uv run publish --task Mjlab-PoliteBow-Flat-MicroDuck \
    --wandb-run-path <entity/project/run_id> --checkpoint 3000 \
    --repo <user>/microduck-polite-bow --kind episodic --duration-s 4.0 \
    --description "Bows from a two-foot stand and comes back up."
```

从已导出的 ONNX 发布（已验证，不再重复导出）：

uv run publish --onnx output.onnx --repo <user>/microduck-flamingo --kind perpetual --unwind-s 1.5 --twist-help "[flag, side, 0]"

为步态槽位发布新的行走策略：

uv run publish --onnx output.onnx --repo <user>/microduck-my-walk --kind perpetual --slot walk

混音/微调策略（基于他人已发布策略时指名父策略，模型卡会写入 base_model 并建立 Hub 关联）：

uv run publish --onnx output.onnx --repo <user>/microduck-deep-bow --kind episodic --duration-s 5.0 --base-model <owner>/microduck-polite-bow

预演（不触碰 Hub，只打印将上传的内容）：

uv run publish --onnx output.onnx --repo <user>/microduck-bow --kind episodic --duration-s 4.0 --dry-run

表 8　--kind 的两种形态与所需参数

形态	参数	说明
episodic	--duration-s（--chain 可选）	运行 --duration-s 秒后自动回到站立姿势（踢球、前滚翻、鞠躬等一次性技巧）；加 --chain 后按住按钮可重复执行
perpetual（步态）	--slot walk（或 stand）	新的行走/站立步态；安装方用 robotctl policy load walk <repo> 装载
perpetual（保持姿势）	--unwind-s（--idle 可选）	如单腿站立：守护进程先用空闲 twist（--idle，默认零）驱动 --unwind-s 秒再交还步态，避免单脚突然放手；安装方用 policy add ... --hold <秒> 作为一次性策略运行

发布前的自动校验：检查计算图是否为 [1,61]→[1,14]（51 维旧版策略会被拒绝）；在合理输入上运行并拒绝 NaN 或常量输出；从 git 与 wandb 填充 training 块（任务、commit、分支、dirty 标志、run、checkpoint）；仓库中已有同名 .onnx 时拒绝覆盖（需 --force）。仓库默认创建为私有；--no-private 可公开，--tag v1 可为版本打标签。

注意：只有常值指令策略可以这种方式发布。相驱动动作（如地面拾取）与姿态标志坐↔站由守护进程本身驱动，属于官方集合 pollen-robotics/microduck-policies。

7.4 真机安装（robotctl）

在真机上（microduck 运行时，需 root 权限）安装发布好的策略：

sudo robotctl policy add polite-bow <user>/microduck-polite-bow # episodic：时长取自 manifest

sudo robotctl policy add flamingo <user>/microduck-flamingo --hold 5 # 保持姿势：自行指定时长

sudo robotctl policy load walk <user>/microduck-my-walk # 步态：装入 walk 槽位

robotctl robot do polite-bow

7.5 渲染回放视频（可选）

可将策略在仿真中的回放渲染为视频（上传后 Hub 的回放组件可播放）：

uv run scripts/render_policy.py output.onnx --seconds 10 --video media/bow.mp4

然后在 publish 时附加该视频：uv run publish --onnx output.onnx --repo <user>/microduck-bow --kind episodic --duration-s 4.0 --video media/bow.mp4。

第 8 章 常见问题排查

表 9 常见问题与处理方法

现象	可能原因	处理方法
uv sync 中途超时失败	网络慢或 ARM 盒子首次拉取约 2 GB CUDA wheels，超过 uv 默认 30 秒 HTTP 超时	首次同步设 UV_HTTP_TIMEOUT=600 uv sync
torch.cuda.is_available() 为 False 或训练报 select_gpus IndexError	aarch64 上装到了 PyPI 的 CPU-only torch	确认 pyproject.toml 中 torch 是直接依赖且为 == 引脚（[tool.uv.sources] 仅对直接依赖生效）；重新 uv sync
训练/冒烟出现 NaN	奖励惩罚项符号错误：自否定惩罚用了负权重，双重否定成奖励	检查任务配置的权重符号；每个 Episode_Reward/<惩罚名> 必须 ≤0
找不到检查点	wandb run 路径或迭代号写错；本地路径不对	核对 --wandb-run-path 为 entity/project/run_id、--checkpoint 为数字迭代号；本地检查点在 logs/<experiment_name>/model_XXXX.pt
手转 checkpoint 部署后行为异常	部署时观测未归一化	必须用 scripts/export.py 导出 ONNX（归一化器已烘焙进图），禁止手转
恢复训练后课程跳到最终阶段	加载了其他任务的检查点且未用 warm start	同任务继续训练用普通 resume；跨任务继承用 MICRODUCK_WARM_START=1（并折叠源课程）

现象	可能原因	处理方法
play 正常但实机异常	sim2real 未对齐	用 infer_policy.py（BAM 与训练一致）演练；用 --vin/--vin-drop-gain/--kp-fw 钉住训练 DR 范围；确认命令观测布局（--new-cmd-obs）一致
观测维度不是 61	误删了指令槽或 height_scan 处理不一致	保留 13 维指令槽（不用则零填充）；velocity 配置已删 height_scan，其余配置照做
uv sync 反复下载失败（tunnel error / tcp connect error / deadline has elapsed）	网络受限：PyPI 直连或代理隧道不稳定	配置清华镜像并重试（见 2.4 节）；必要时先执行 unset http_proxy https_proxy HTTP_PROXY HTTPS_PROXY ALL_PROXY；失败后重跑可续传缓存

此外，训练实践中的几条经验：若某条 wandb 指标恰好在课程阶段边界处下降，说明课程节奏不对（应拉长阶段或延后引入，绝不提前）；反向前置课程（从动作中途开始生成回合）是解决“只学会开头、学不会最后一步”的可靠手段。当一次 run “失败”时，先对实际检查点做无头评估再改奖励——过往许多“失败”实为检查点过早、成功判据把同一行为簇一分为二、或奖励上限与实测物理冲突。评估以实际 rollout 表现为准（例如“能滚翻但三次里有一次脸着地”），由用户决定何时算足够好。

第 9 章 命令速查表

表 10 命令速查表

目的	命令
查看任务注册表	uv run list-envs
冒烟测试	uv run train <TASK_ID> --env.scene.num-envs 64 --agent.max_iterations 5
正式训练	uv run train Mjlab-Velocity-Flat-MicroDuck --env.scene.num-envs 4096
云端训练（无 GPU）	uv run train <TASK_ID> --env.scene.num-envs 4096 --hf-jobs
恢复训练	uv run train Mjlab-Velocity-Flat-MicroDuck --env.scene.num-envs 4096 --agent.run-name resume --agent.load-checkpoint model_XXXX.pt --agent.resume True
跨任务热启动	MICRODUCK_WARM_START=1 uv run train <TASK_ID> --env.scene.num-envs 4096 --agent.resume True --wandb-run-path <entity/project/run_id> --wandb-checkpoint-name model_XXXX.pt
查看器回放	uv run play <TASK_ID> --wandb-run-path <entity/project/run_id>
部署演练	uv run scripts/infer_policy.py --walking output.onnx --new-cmd-obs
导出 ONNX	uv run scripts/export.py<TASK_ID> --wandb-run-path <...>

目的	命令
渲染回放视频	uv run scripts/render_policy.py output.onnx --seconds 10 --video media/out.mp4
发布策略	uv run publish --task <TASK_ID> --wandb-run-path <...> --checkpoint N --repo <user>/microduck-<name> --kind episodic --duration-s 4.0
回归测试	uv run --with pytest pytest tests/

附录 A 仓库关键文件与进一步阅读

以下文件是深入了解训练配置与经验的入口：

表 11 仓库关键文件

文件	内容
README.md	快速开始、任务清单、Backlash、执行器模型、机器人模型、发布说明
AGENTS.md	训练经验与规则（奖励设计符号约定、读 run 方法、sim2real 脚坑、预算）
src/mjlab_microduck/tasks/microduck_velocity_env_cfg.py	主行走配方：域随机化、观测、命令、PPO 超参（其他任务配置以此为基础）
src/mjlab_microduck/tasks/mdp.py	全部自定义奖励、事件、观测、命令与课程函数
src/mjlab_microduck/tasks/backlash.py	Backlash 变体环境包装
src/mjlab_microduck/actuator/friction_dr_bam.py	BAM 执行器 + 摩擦域随机化 + 齿隙编码器反馈
src/mjlab_microduck/export.py	ONNX 导出（归一化器烘焙）
src/mjlab_microduck/publish/	publish 命令：manifest 构建、ONNX 校验、Hub 上传
scripts/hf/README.md	Hugging Face Jobs 提交与参数说明
docs/velstand_policy.md、docs/sitstand_policy.md	真实策略训练履历与 warm start 示例
tests/	配置不变性与奖励函数回归测试

相关项目：microduck（机器人项目主页与机载运行时，https://github.com/pollen-robotics/microduck）、mjlab（训练框架，https://github.com/mujocolab/mjlab）、BAM（执行器模型库，https://github.com/Rhoban/bam）。

8 浏览器仿真工具（可视化理解架构运行）

无需本地部署，直接在浏览器体验架构闭环：

地址：https://huggingface.co/spaces/pollen-robotics/microduck-simulator

入手点：

- 操作交互：用 WASD / 游戏手柄控制机器人（行走、踢球、轮滑），直观感受「输入指令→策略推理→电机执行」的链路；
- 核心观察：切换「腿部 / 轮滑模式」（按 M 键），理解不同 ONNX 策略如何被加载和执行；
- 调试模式：添加 URL 参数 ?boot=1 进入 BIOS 控制台，查看模型加载、策略初始化的实时日志（对应硬件端的启动流程）。

9 官方文档与特性说明（补充架构设计理念）

- 从 GitHub 仓库的 docs/ 目录入手，重点看 agents.md（强化学习策略设计手册）；
- 参考产品页面技术说明：https://pollen-robotics.com/microduck/ （明确软件与硬件的交互边界，如传感器数据格式、电机控制协议）。

10 RL 算法

10.1 强化学习到底在解决什么问题？

累计回报：$G_t = R_{t+1} + \gamma R_{t+1} + \gamma^2 R_{t+3} + \ldots\ldots$

即：

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \qquad \text{公式 1}$$

其中：

- R_{t+k+1}：执行动作之后得到的即时奖励
- $\gamma \in [0,1]$：折扣因子
- G_t：从时刻 t 开始得到的总回报。

该公式的物理含义：从公式 1 看，累计回报就是从此时此刻开始后续动作的回报的和。更直观地说：

时刻 t 只是一个起点，当前选择动作A_t，到达下一个状态后获得奖励R_{t+1}，再执行下一步动作获得R_{t+2}，再下一步获得R_{t+3}，一直累加下去。

因此可以理解为：G_t =未来所有奖励的总和。只是因为未来越远越不确定，所以强化学习通常乘以折扣因子 γ，让远期奖励的影响变小。

强化学习比较的不是某一步的奖励，而是多条路径各自对应的累计回报G_t，然后选择G_t更大的策略。

所以强化学习真正优化的并不是：

“下一步奖励最大。”

而是：

“从现在开始，长期累计得到的奖励最大。”

这就是强化学习和很多普通监督学习问题最根本的区别之一。

10.2 为什么需要“状态、动作、奖励”？

强化学习通常用一个数学对象描述环境：

$$\mathrm{MDP} = (\mathrm{S}, \mathrm{A}, \mathrm{P}, \mathrm{R}, \gamma)$$

其中：

- S：状态空间

- A：动作空间

- P：状态转移概率

- R：奖励函数

- γ：折扣因子

状态转移概率 $\mathrm{P}(S'|\mathrm{s},\mathrm{a})$ 公式 2

公式 2 表示，当前处于状态 s，采取动作 a 后，下一状态变成 s'的概率。

例如，$\mathrm{P}(S'_{左}|\mathrm{s}_{中},\mathrm{a}_{左}) = 0.9$，表示：当前在中间位置，选择向左走，有 90%的概率真的到达左边。

物理意义：这就是环境的动力学规律。

例如机器人：当前位置+控制力⟶下一位置，但现实世界存在噪声，所以不是确定性的。例如机器人想向前走，尽管采取的动作都是向前，但实际可能向前、向左、向右（因为机械结构误差、控制误差等等因素）：

$$\mathrm{P}\left(\text{向前}\middle|\mathrm{s}, a_{\text{前进}}\right) = 0.95$$

$$\mathrm{P}\left(\text{向左}\middle|\mathrm{s},a_{\text{前进}}\right)=0.03$$

$$\mathrm{P}\left(\text{向右}\middle|\mathrm{s},a_{\text{前进}}\right)=0.02$$

因此强化学习本质上是在一个可能存在随机性的动态系统中做决策。

10.3 策略是什么？

策略：　　$\pi(\mathrm{a}|\mathrm{s})$　　公式 3

数学意义：

$$\pi(\mathrm{a}|\mathrm{s})=\mathrm{P}(A_t=\mathrm{a}|S_t=\mathrm{s})$$

它表示：已知当前状态 s，选择动作 a 的概率。

例如：

$$\pi(\mathrm{a}_{\text{左}}|\mathrm{s})=0.8$$

$$\pi\left(\mathrm{a}_{\text{右}}\middle|\mathrm{s}\right)=0.2$$

说明机器人在这个状态下：有 80%概率采取向左的动作，另有 20%概率采取向右的动作。

物理意义：策略就是机器人的行为规则，公式 3 **就是表示在当前状态下采取什么行动。**也可以理解成一个控制器：

$$\mathrm{s}\longrightarrow\mathrm{a}$$

比如：看到障碍物⟶向左转。

强化学习的核心任务之一就是：寻找最优策略。

10.4 为什么需要 VALUE FUNCTION？

我们已经知道：$\boldsymbol{G_t}$表示从现在开始最终能得到多少奖励。

但问题来了：如果现在处于状态 s，未来存在随机性，那么这个$\boldsymbol{G_t}$每次可能都不同。所以我们考虑它的**数学期望**。

$$V^{\pi}(s)=E_{\pi}[G_t|S_t=s]$$　　公式 4

它的含义是在状态 s 出发，并按照策略π行动（注意：是策略而不是行动），未来累计回报的平均值。注意这里非常重要：

$$V^{\pi}(s)\neq G_t$$

而是：

$$V^{\pi}(s)=E[G_t]$$

G_t是一次具体经历得到的回报，$V^{\pi}(s)$是大量可能经历的平均回报。

物理意义：可以把$V^{\pi}(s)$理解成：“这个位置（状态）到底值多少钱？”

例如：

状态	$V^{\pi}(s)$
状态 1（位置离终点一步）	+9
状态 2（位置离终点三步）	+6
状态 3（位置靠近陷阱）	−8

那么机器人看到不同位置，就可以判断：“这个状态好不好？”这就是 Value Function。

10.5 BELLMAN 方程：强化学习最核心的数学结构

这是整个强化学习数学体系最重要的公式之一。

Bellman Expectation Equation

从：

$$G_t = R_{t+1} + \gamma G_{t+1} \qquad \text{公式 5}$$

开始，两边取条件期望，就是：

$$V^{\pi}(s) = E_{\pi}[R_{t+1} + \gamma V^{\pi}(S_{t+1}) | S_t = s]$$

这个公式的意思实际上只是把一个长期问题拆成：

现在得到多少+未来值多少

也就是：

$$V^{\pi}(s) = \frac{\textit{即时奖励}}{\textit{现在}} + \gamma \frac{\textit{未来价值}}{\textit{未来}}$$

这就是 Bellman 思想。

为什么它这么重要？

因为原来的问题：$G_t = R_{t+1} + \gamma R_{t+1} + \gamma^2 R_{t+3}$+……是一个很长的未来序列。Bellman 方程把它压缩成：{当前一步}+{下一状态的价值}，即：

$$G_t = R_{t+1} + \gamma V(s')$$

这叫做：Bellman decomposition（贝尔曼分解）。

物理意义：

可以想象你站在一个岔路口。你不需要把未来 1000 步全部模拟出来。只需要问：

“走这一步，我马上获得什么？走过去之后，那个位置又值多少钱？”

于是：

{当前价值}={当前收益}+{未来价值}

这实际上就是强化学习能够进行计算的根本原因。

10.6 Q FUNCTION：为什么还需要它？

Value Function 只告诉我们：V(s)，也就是：“这个状态好不好？”。但是我们真正需要做决策时，更关心：“在这个状态下，我采取哪个动作最好？”

于是定义 Q 函数（动作价值函数）。

$$Q^{\pi}(s,a) = E_{\pi}[G_t|S_t = s, A_t = a] \quad \text{公式 6}$$

数学意义

它计算：已知现在处于 s，并且现在强制采取动作 a，之后继续按照策略π行动，未来总回报的期望。

所以V(s)回答：这个状态怎么样？而 Q(s,a)回答：在这个状态下做这个动作怎么样？

物理意义

例如机器人处于：s={岔路口}

那么：Q(s,左)表示：

如果现在向左走，未来总体有多好？而：Q(s,\右)表示：

如果现在向右走，未来总体有多好？

于是决策非常自然：

$$a^* = \underset{a}{argmax}\, Q(s,a)$$

也就是：

选择 Q 值最大的动作。

10.7 V 和 Q 的关系

这是另一个非常重要的公式。

$$V^{\pi}(s) = \sum_{a} \pi(a|s) Q^{\pi}(s,a) \quad \text{公式 7}$$

数学意义：这是一个加权平均。假设：

Q(s,左)=10

Q(s,右)=2

而策略：

π(左|s)=0.7

π(右|s)=0.3

那么：

V(s)=0.7(10)+0.3(2)=7.6

物理意义可以理解成：

状态价值 = 按照当前行为习惯做各种动作后，得到的平均结果。因此 Q(s,a)像是：“这个动作的成绩。”，而 V(s)像是：“按照当前策略综合起来，这个位置的平均成绩。”

10.8 最优价值函数

现在我们不再满足于某一个策略。我们希望：所有可能策略中，哪个最好？

$$V^*(s) = \max_{\pi} V^{\pi}(s) \qquad \text{公式 8}$$

数学意义：在所有可能策略中取最大值。也就是：

$$V^*(s) = \max_{\pi} E_{\pi}[G_t | S_t = s]$$

物理意义：它表示，如果机器人足够聪明，从这个状态出发，它最多能够获得多少长期收益？所以$V^*(s)$是一个“理论上的最高分”。

10.9 最优 Q 函数

$$Q^*(s,a) = \max_{\pi} Q^{\pi}(s,a) \qquad \text{公式 9}$$

数学意义

在当前状态 s 下，先执行动作 a，然后未来采用最优策略。

因此：Q*(s,a)就是这个动作在最优条件下能够达到的最大期望回报。

物理意义

它就是：“现在做这个动作到底值不值得？”于是最优动作：

$$\pi^*(s) = \arg\max_{a} Q^*(s,a)$$

这句话非常重要：

强化学习最终就是在寻找一个能够最大化长期回报的策略。

10.10 BELLMAN OPTIMALITY EQUATION

把前面的思想结合起来。

$$V^*(s) = \max_{a} E[R_{t+1} + \gamma V^*(S_{t+1}) | S_t = s, A_t = a] \qquad \text{公式 10}$$

数学意义

它告诉我们：最优价值 $= \max_{\text{动作}}$[即时奖励] $+ \gamma$[*未来最优价值*]

注意这里出现了 max，这意味着：不再按照一个固定策略平均，而是在动作中选择最优的。

物理意义

举例说明，比如机器人来到岔路口：

左：{立即奖励}+{未来价值}

右：{立即奖励}+{未来价值}

机器人选择其中更大的那个。所以：

{今天的最优选择取决于明天的最优价值}

这就是动态规划的核心思想。

10.11 为什么 Q-LEARNING 如此重要？

Q-learning 直接学习：$Q^*(s,a)$，而不需要显式学习 V(s)。

它的核心更新公式是：

Q-learning: $Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha[r_{t+1} + \gamma \max_{\alpha'} Q(s_{t+1},\alpha') - Q(s_t,\alpha_t)]$ 公式 11

这是强化学习中最经典的公式之一。

先看括号里的东西，定义：

$$\delta_t = r_{t+1} + \gamma \max_{\alpha'} Q(s_{t+1},\alpha') - Q(s_t,\alpha_t)$$

这叫 TD Error，时序差分误差。

数学意义：$\delta_t = 目标值 - 当前估计$，其中目标：$r_{t+1} + \gamma \max_{\alpha'} Q(s_{t+1},\alpha')$，当前估计：$Q(s_t,\alpha_t)$，所以：$\delta =$ 新信息 $-$ 旧预测。然后$Q_{new} = Q_{old} + \alpha\delta$。

物理意义

假设机器人原来认为：“向左走很好，价值 8。”，所以：Q(s,左)=8。

结果机器人走过去之后发现：原来前面竟然有一个奖励+n 的终点。所以实际发现：$r_{t+1} + \gamma \max_{\alpha'} Q(s_{t+1},\alpha') = 12$，于是：$\delta = 12 - 8 = 4$

如果$\alpha = 0.1$，那么：Qnew=8+0.1(4)=8.4

机器人不会一下子认为：Q=12，而只是：8->8.4，逐渐修正。

10.12 TD LEARNING 的本质

我们可以把 Q-learning 的更新写成：{新估计}={旧估计}+{学习率}{预测误差}，即：

$$Q \leftarrow Q + \alpha\delta$$

数学意义：这其实非常像很多数值优化算法：$\theta_{new} = \theta_{old} + \text{correction}$

也就是说：强化学习不断利用预测误差修正自己的函数估计。

物理意义：就像一个人在学走路。他预测：“踩这里应该没问题。”。结果：“啪，摔倒了。”，于是：{预测}->{实际反馈}->{误差}->{修正}。下一次：“这里不能踩。”这就是 TD Learning 最直观的物理图像。

10.13 探索与利用

到这里有一个非常现实的问题。假设机器人现在知道：Q(s,左)=8、Q(s,右)=5。它当然会选择：{左}。但是：如果它从来没有尝试过右边，又怎么知道右边是不是实际上有 +100 的宝藏？这就是：Exploration vs Exploitation（探索 vs 利用）。

$\epsilon - \text{greedy}$ $\quad \pi(a|s) = \begin{cases} \text{随机动作}, \epsilon \\ argmax_a Q(s,a), 1-\epsilon \end{cases}$ 公式 12

数学意义：以概率ϵ进行随机探索，以$1-\epsilon$的概率选择当前最优动作。例如：$\epsilon = 0.1$，则意味着：90% 时间利用当前知识，10% 时间探索未知动作。

物理意义：就像一个人在餐厅吃饭：如果你一直点自己最喜欢的菜：永远点牛肉面。你可能永远不知道：新菜单上的鱼香肉丝其实更好吃。所以偶尔要：“试一下没吃过的。”这就是探索。

10.14 从表格 Q 到神经网络

前面的 Q-learning 有一个隐含假设：状态数量不是特别大。可以直接建立：Q(s,a)表格。

例如：

状态	左	右
s_1	5	2
s_2	8	4
s_3	1	9

但是如果状态是图像，比如 84*84*3，那么状态空间巨大，根本无法建立 Q 表。

于是我们用神经网络近似：$Q(s,a,\theta)$，其中θ是神经网络参数。

10.15 强化学习突然变成了函数逼近问题

$$Q(s,a) \approx Q(s,a,\theta)$$ 公式 13

数学意义：我们不再存储每一个状态的 Q 值。而是寻找一个函数：

$$f_\theta(s,a)$$

使得：

$$f_\theta(s,a) \approx Q^*(s,a)$$

所以强化学习变成了：寻找一个参数θ，使神经网络能够逼近最优价值函数。θ即神经网络的权值。

物理意义：过去机器人需要记忆：“这个位置是 8 分。”现在机器人学习的是一个规律：“距离终点越近、离陷阱越远，通常价值越高。”因此它从：{记忆}变成：{泛化}。

10.16 DEEP Q-NETWORK 的损失函数

Q-learning 可以构造一个目标：$y = r + \gamma \max_{a'} Q(s', a', \theta')$

然后希望：$Q(s, a, \theta) \approx y$

所以定义损失：

$$L(\theta) = \frac{1}{2}[y - Q(s, a, \theta)]^2 \quad \text{公式 14}$$

数学意义：这是普通的平方误差$Loss = \frac{1}{2}(\textit{目标} - \textit{预测})^2$，然后用梯度下降：

$$\theta \leftarrow \theta - \eta \nabla_\theta L(\theta)$$

即神经网络不断调整参数，使预测越来越接近经验目标。

10.17 POLICY GRADIENT：为什么可以直接学策略？

前面我们学习：Q(s,a)，再从 Q 中选择动作。另一条路线是直接学习：$\pi_\theta(a|s)$，也就是：直接让神经网络决定应该采取什么动作。

$$J(\theta) = E_{\tau \sim \pi_\theta}[G(\tau)] \quad \text{公式 15}$$

其中τ表示一整条轨迹：

$$\tau = (s_0, a_0, r_1, s_1, \dots)$$

数学意义：我们希望J(θ)最大。也就是：让策略产生的轨迹，其平均累计奖励最大。即：$\max_\theta J(\theta)$。

物理意义：这次我们不再问“哪个动作价值最高？”，而是直接问：“怎样调整我的行为习惯，才能让我以后获得更多奖励？”。例如机器人原来：P(左)=0.5，P(右)=0.5。经过训练后：P(左)=0.9，P(右)=0.1，它直接改变自己的行为概率。

10.18 POLICY GRADIENT 最核心的公式

$$\nabla_\theta J(\theta) = E[\nabla_\theta log \pi_\theta(a|s) G_t] \quad \text{公式 16}$$

这是著名的 Policy Gradient / REINFORCE 形式。

数学意义：如果某次动作最终带来了很高的：G(t)，那么：$\nabla_\theta log \pi_\theta(a|s)$会推动神经网络以后更倾向于选择这个动作。反之，如果：G(t)很低，那么这个动作会被抑制。

物理意义：可以想象训练一个小孩。小孩做了一个决定：“我要走左边。”结果：+100。你告诉他：“很好，以后多这样做。”于是：P(左)。如果：-100，你告诉他：“这条路不好，以后少这么做。”于是：P(左)。

Policy Gradient 本质上就是：{奖励行为强化，坏行为削弱}

10.19 Actor-Critic：价值函数和策略函数结合

现在我们有两套思想：

Critic：学习：V(s)，或者 Q(s,a)，它负责评价行为。

Actor：学习：π(a|s)，它负责选择行为。

于是：$\boxed{\text{Actor}\} + \text{Critic}}$

优势函数 $$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$$ 公式 17

数学意义：它比较采取动作后的价值$Q^{\pi}(s,a)$和这个状态的一般价值$V^{\pi}(s)$。如果：$A^{\pi}(s,a) > 0$。说明这个动作比平均水平好。如果$A^{\pi}(s,a) < 0$，说明这个动作比平均水平差。

优势函数就是在问：“这个动作到底比这个状态下的平均水平好多少？”

10.20 Actor-Critic 的直觉

Actor：π_{θ}(a|s)，负责“我应该怎么做？”。

Critic：V(s) 负责评价“你做得怎么样？”

于是：{Actor 决策}->{环境反馈}->{Critic 评价}->{Actor 改进}

这就是现代强化学习很多算法的基本结构。

10.21 把整个强化学习数学体系串起来

现在可以把前面的公式连成一条链。

最开始：

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

定义状态价值：$V^{\pi}(s) = E[G_t | S_t = s]$

定义动作价值：$Q^{\pi}(s,a) = E[G_t | S_t = s, A_t = a]$

得到 Bellman 方程：$V^{\pi}(s) = E[R + \gamma V^{\pi}(s')]$

最优情况下：$V^{*}(s) = max_a E[R + \gamma V^{*}(s')]$

进一步：$Q^{*}(s,a) = E[R + \gamma max_{a'} Q^{*}(s',a')]$

于是可以进行 Q-learning：$Q \leftarrow Q + \alpha[r + \gamma max_{a'} Q' - Q]$

如果状态空间巨大：$Q(s,a) \rightarrow Q(s,a;\theta)$

再用神经网络优化：$\theta \leftarrow \theta - \eta \nabla_\theta L$

另一条路线则直接优化：$max_\theta J(\theta)$

得到：$\nabla_\theta J = E[\nabla_\theta \log \pi_\theta(a|s) \cdot G_t]$

最后形成：Actor-Critic

10.22 如果用一句话理解整个强化学习

可以把强化学习浓缩成：尝试动作→获得奖励→评价结果→修正策略→再次尝试。

数学上就是：Policy→Trajectory→Return→Value→Error→Update。

也就是：$\pi \rightarrow G \rightarrow V/Q \rightarrow \delta \rightarrow \theta \rightarrow \pi_{new}$，这实际上就是强化学习的数学闭环。

最重要的“物理图像”：如果只记住一张图，可以记：状态 s 经 $\pi(a|s)$ 选择动作 a，环境返回奖励 r 与新状态 s′；然后 (s,a,r,s′) 计算误差，更新价值与策略。

最终目标：$max_\pi E_\pi[\sum_{t=0}^{\infty} \gamma^t R_{t+1}]$，这就是强化学习最根本的数学目标。